

Provider-Agnostic Wallet Base Architecture — Proposal Implementation Plan

RFP: RFP-T-BACK-0006 v2.3.1

Scope: WBS-BASE-01 through WBS-BASE-26 (67 engineering hours baseline)

Current System Summary

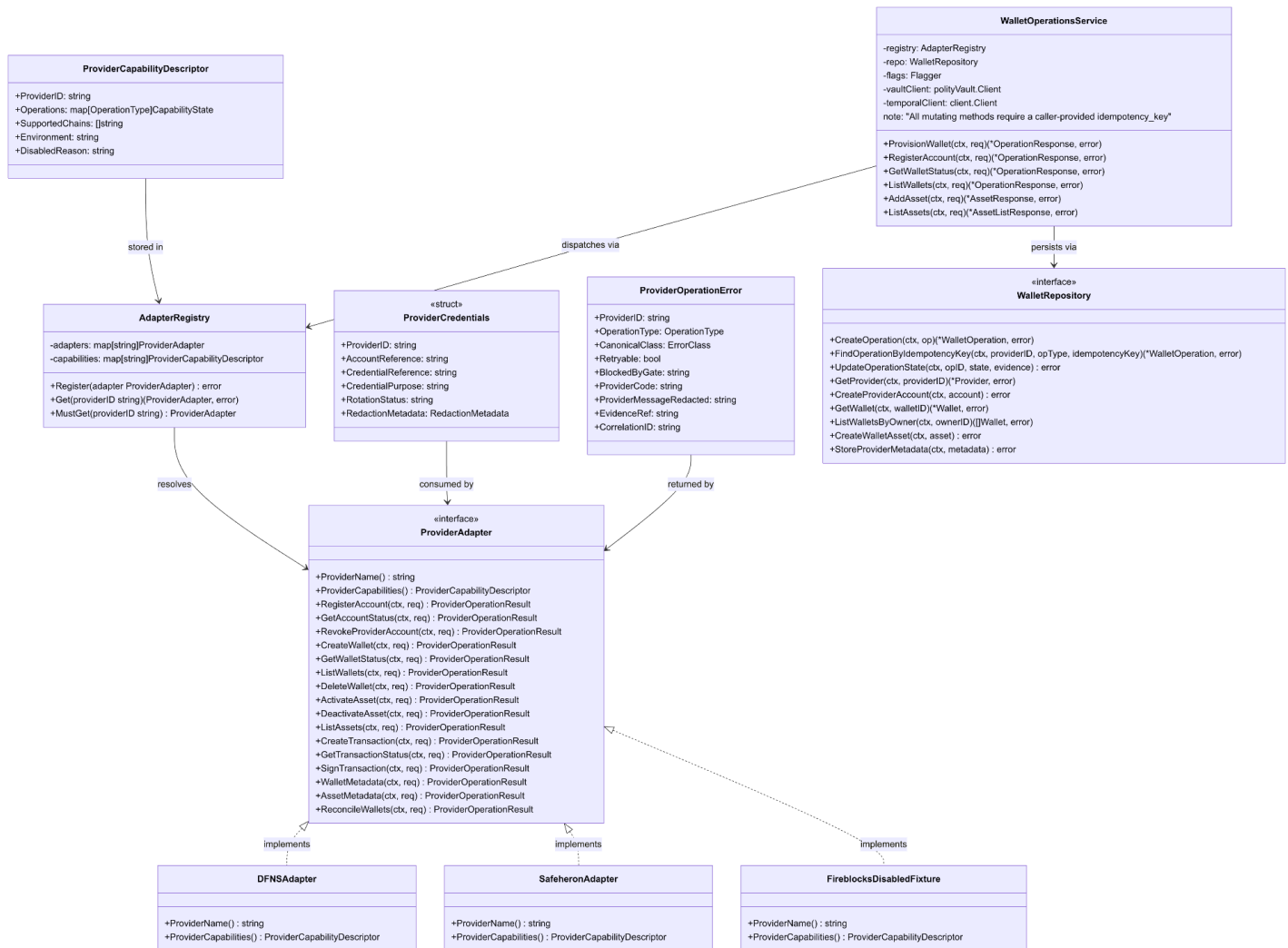
The existing `wallet-infrastructure-service` is a Go monolithic service (fasthttp + GORM + gormigrate + Redis) that tightly couples DFNS and Safeheron providers:

- `internal/rest/api/api.go` — 2000+ line monolithic controller with 16+ injected provider-specific dependencies
- `internal/service/interface.go` — Generic `WalletInfrastructureService[T vault.DataConstraints]` with payment/signing/broadcast methods mixed in
- `internal/service/wallets/dfns/` and `internal/service/wallets/safeheron/` — Concrete provider implementations with full SDK coupling
- `internal/rest/router/router.go` — Provider-specific route pairs (`/wallets/dfns/...`, `/wallets/safeheron/...`)
- `providers/polityVault/client.go` — Vault REST gateway with credential fetching + Redis caching (provider-specific: `GetSafeheronWalletCredentials`, `GetDFNSWalletCredentials`)
- `polity_vault/vault_backend/client.py` — Python-side Infisical client (`BaseInfisicalClient ABC`, `RealInfisicalClient`, `MockInfisicalClient`) that the vault REST service uses internally to fetch secrets from Infisical; WIS never calls Infisical directly

- `internal/storage/postgres.go` — Provider-specific table sets (`dfns_assets`, `safeheron_assets`)
- `internal/storage/migrations/migrations.go` — 11 gormigrate steps with provider-specific migrations
- `cmd/main.go` — Single binary entrypoint, no worker process, no Temporal integration

Target: Introduce provider-agnostic base contract, AdapterRegistry, Temporal workflows, WALLET_OPERATIONS ledger, unified API, and disabled-operation gates while preserving existing DFNS/Safeheron read behaviour through Strangler Fig migration.

Overall Architecture — Target State



MR-BASE Package Overview

MR Package	Branch	WBS Rows	Purpose
MR-BAS E-01	<code>feature/wallet-base/base-01-inventory-contract-freeze</code>	WBS-BA SE-01..03	Inventory + contract freeze
MR-BAS E-02	<code>feature/wallet-base/base-02-contract-registry-flags</code>	WBS-BA SE-04..08	Interfaces, registry, errors, flags, vault contract
MR-BAS E-03	<code>feature/wallet-base/base-03-persistence-idempotency</code>	WBS-BA SE-09..13	Persistence, seed, operations ledger, idempotency, rollback
MR-BAS E-04	<code>feature/wallet-base/base-04-temporal-vault-boundary</code>	WBS-BA SE-14..18	Temporal client, worker, workflows, activities, saga
MR-BAS E-05	<code>feature/wallet-base/base-05-api-compatibility-closure</code>	WBS-BA SE-19..23	Unified API, route fate, negative proof, compatibility, callbacks
MR-BAS E-06	<code>feature/wallet-base/base-06-evidence-provider-handoff</code>	WBS-BA SE-24..26	Traceability, runbook, final QA

Effort Summary

MR Package	WBS Rows	Individual Hours	Total Hours
MR-BASE-01	WBS-BASE-01 (2h) + WBS-BASE-02 (1h) + WBS-BASE-03 (1h)	2 + 1 + 1	4h
MR-BASE-02	WBS-BASE-04 (2h) + WBS-BASE-05 (2h) + WBS-BASE-06 (2h) + WBS-BASE-07 (2h) + WBS-BASE-08 (1h)	2 + 2 + 2 + 2 + 1	9h
MR-BASE-03	WBS-BASE-09 (3h) + WBS-BASE-10 (2h) + WBS-BASE-11 (3h) + WBS-BASE-12 (2h) + WBS-BASE-13 (1h)	3 + 2 + 3 + 2 + 1	11h
MR-BASE-04	WBS-BASE-14 (2h) + WBS-BASE-15 (2h) + WBS-BASE-16 (2h) + WBS-BASE-17 (2h) + WBS-BASE-18 (2h)	2 + 2 + 2 + 2 + 2	10h
MR-BASE-05	WBS-BASE-19 (2h) + WBS-BASE-20 (2h) + WBS-BASE-21 (2h) + WBS-BASE-22 (1h) + WBS-BASE-23 (2h)	2 + 2 + 2 + 1 + 2	9h
MR-BASE-06	WBS-BASE-24 (2h) + WBS-BASE-25 (2h) + WBS-BASE-26 (3h)	2 + 2 + 3	7h

		Grand Total	50h
--	--	-------------	-----

WBS-BASE-01: Route and Sensitive-Path Inventory

Time: 2h | MR: MR-BASE-01 | Dependencies: G0, AC-05/G2, AC-07/G5, AC-10/G7

Files to Examine and Document

File	Action	Description
<code>internal/rest/router/router.go</code>	Inventory	All registered routes with HTTP methods and middleware
<code>internal/rest/api/api.go</code>	Inventory	All handler methods, identify sensitive-operation handlers
<code>internal/rest/router/middleware.go</code>	Inventory	JWT, RBAC, webhook verification middleware

Sensitive-Path Classification

Route classification from `router.go` (current routes):

Route	Method	Classification	Fate
<code>/api/v1/wallet/dfns</code>	POST	Payment/Transaction — creates DFNS wallet via <code>CreateDFNSWallet</code>	Wrap (behind gate, then migrate)
<code>/api/v1/wallet/safeheron</code>	POST	Payment/Transaction — creates Safeheron wallet via <code>CreateSafeheronWallet</code>	Wrap (behind gate, then migrate)
<code>/api/v1/wallets/dfns</code>	GET	Read — list DFNS wallets	Preserve (compatibility bridge)
<code>/api/v1/wallets/safeheron</code>	GET	Read — list Safeheron wallets	Preserve (compatibility bridge)
<code>/api/v1/wallet/:id</code>	GET	Read — get single wallet	Preserve
<code>/api/v1/wallet/:id/rename</code>	PUT	Write — rename wallet	Wrap (behind gate)
<code>/api/v1/wallet/:id/archive</code>	PUT	Write — archive wallet	Wrap (behind gate)

<code>/api/v1/wallets/dfns/assets</code>	GET	Read — list DFNS assets	Preserve (compatibility bridge)
<code>/api/v1/wallets/safeheron/assets</code>	GET	Read — list Safeheron assets	Preserve (compatibility bridge)
<code>/api/v1/wallet/dfns/asset</code>	POST	Write — create DFNS asset	Wrap (behind gate)
<code>/api/v1/wallet/safeheron/asset</code>	POST	Write — create Safeheron asset	Wrap (behind gate)
<code>/api/v1/wallet/dfns/asset/archive</code>	PUT	Write — archive DFNS asset	Wrap (behind gate)
<code>/api/v1/wallet/safeheron/asset/archive</code>	PUT	Write — archive Safeheron asset	Wrap (behind gate)
<code>/api/v1/wallet/address</code>	GET	Read — get asset addresses	Preserve
<code>/api/v1/payment/dfns</code>	POST	SENSITIVE: Payment	Disable (kill-switch gate, not reachable)

<code>/api/v1/payment/resolve</code>	POST	SENSITIVE: Payment resolution	Disable
<code>/api/v1/payment/safeheron</code>	POST	SENSITIVE: Payment	Disable
<code>/api/v1/wallet/dfns/trans actions</code>	GET	Read — get DFNS transactions	Preserve (compatibility)
<code>/api/v1/wallet/safeheron/ transactions</code>	GET	Read — get Safeheron transactions	Preserve (compatibility)
<code>/api/v1/defi/products</code>	GET	Read — list DeFi products	Preserve
<code>/api/v1/dfns/notification</code>	POST	SENSITIVE: Webhook callback	Disable (behind envelope)
<code>/api/v1/safeheron/notific ation</code>	POST	SENSITIVE: Webhook callback	Disable (behind envelope)
<code>/api/v1/walletconnect/inv oke</code>	POST	SENSITIVE: WalletConnect invocation	Disable (kill-switch)

<code>/api/v1/wallet/scan/dfns</code>	POST	SENSITIVE: Transaction scan	Disable
<code>/api/v1/wallet/scan/safeheron</code>	POST	SENSITIVE: Transaction scan	Disable
<code>/api/v1/favourite/address</code>	GET/POST/P UT/DELETE	Read/Write — address book	Preserve (compatibility)
<code>/health</code>	GET	Read — health check	Preserve

Sensitive-Operation Register

```
// internal/contract/sensitive_paths.go — NEW FILE (inventory only, no runtime
activation)
package contract

// SensitivePathRegistry documents all wallet routes classified by fate.
// This is an inventory document; runtime gating is implemented in WBS-BASE-07.
var SensitivePathRegistry = []RouteEntry{
    {Method: "POST", Path: "/api/v1/payment/dfns", Category: "payment", Fate:
"disable"},
    {Method: "POST", Path: "/api/v1/payment/safeheron", Category: "payment",
Fate: "disable"},
    {Method: "POST", Path: "/api/v1/payment/resolve", Category: "payment",
Fate: "disable"},
    {Method: "POST", Path: "/api/v1/walletconnect/invoke", Category:
"walletconnect", Fate: "disable"},
    {Method: "POST", Path: "/api/v1/wallet/scan/dfns", Category: "scan", Fate:
"disable"},
    {Method: "POST", Path: "/api/v1/wallet/scan/safeheron", Category: "scan",
Fate: "disable"},
    {Method: "POST", Path: "/api/v1/dfns/notification", Category: "webhook",
Fate: "disable"},
    {Method: "POST", Path: "/api/v1/safeheron/notification", Category:
"webhook", Fate: "disable"},
}
```

```
// ... additional entries
}

type RouteEntry struct {
    Method string
    Path    string
    Category string
    Fate    string // "preserve", "wrap", "disable", "deprecate", "remove"
}
```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Static route scan of <code>router.go</code>	Route inventory	Generate by enumerating all <code>router.{METHOD}</code> calls
Endpoint list with categories	Sensitive-path register	Map each route to payment/signing/broadcast/webhook/scan/admin categories
Exception log	Exception log	Document any routes that cannot be immediately classified
Reviewer sign-off	Sensitive paths removed/disabled/gated	This is a documentation-only WBS item; no code changes to routes yet

WBS-BASE-02: Persistence and Credential Inventory

Time: 1h | MR: MR-BASE-01 | Dependencies: G0, AC-03/G3-G4, AC-04/G6

Files to Examine

File	Action	Description
<code>internal/storage/postgres.go</code>	Inventory	All GORM model definitions and queries
<code>internal/storage/migrations/migrations.go</code>	Inventory	All 11 gormigrate migration steps
<code>internal/models/polity/wallet.go</code>	Inventory	Wallet model
<code>internal/models/dfns/</code>	Inventory	DFNS-specific models (Wallet, Asset)
<code>internal/models/safeheron/</code>	Inventory	Safeheron-specific models (Asset, Wallet, Callback, Coin, Payment)

<code>internal/models/vault/vault.go</code>	Inventory	Vault credential models (SafeheronCredentials, DFNSCredentials)
<code>providers/polityVault/client.go</code>	Inventory	Vault REST gateway interface and implementation
<code>providers/polityVault/mock.go</code>	Inventory	Mock vault client

Table Inventory

Current Table	GORM Model	Provider-Specific?	Credential Material?	Migration to Provider-Neutral
<code>polity.Wallet</code>	<code>polity.Wallet</code>	No (generic)	No	Add <code>provider_id</code> , <code>provider_account_id</code> , <code>normalised_status</code>
<code>dfns_wallets</code> (was <code>dfns_assets</code> → <code>dfns_wallets</code>)	<code>dfns.Wallet/dfns.Asset</code>	Yes (DFNS)	No	Read-through compatibility view during Strangler Fig

<code>safeheron_assets</code>	<code>safeheron.Asset</code>	Yes (Safeheron)	No	Read-through compatibility view
<code>favourite_address_entries</code>	<code>polity.FavouriteAddressEntry</code>	No	No	Preserve as-is
<code>defi_products</code>	<code>defi.DefiProduct</code>	No	No	Preserve as-is

Credential Boundary

```
// providers/polityVault/client.go – EXISTING (inventory, no code changes in this WBS)
//
// Current polityVault.Client interface (provider-specific methods):
//   GetSafeheronWalletCredentials(token, cosigner_uuid, options) → *vault.SafeheronCredentials
//   GetDFNSWalletCredentials(token, options) → *vault.DFNSCredentials
//   CreateWalletRequest(request, token) → error
//
// CURRENT ARCHITECTURE:
//   WIS (Go) → polityVault.Client → HTTP REST → polity_vault (Python/Django)
//                                     ↓
//                                     vault_backend/client.py
//                                     (RealInfisicalClient /
MockInfisicalClient)
//                                     ↓
//                                     Infisical API (secrets backend)
//
// KEY FINDINGS:
//   1. WIS does NOT call Infisical directly. All credential access goes through
//       the Go polityVault.Client → Python polity_vault REST API → Infisical.
//       The RFP requirement "WIS must not call Infisical directly" is ALREADY satisfied.
```

```
//
// 2. polity_vault/vault_backend/client.py (Python) is the Infisical
integration point.
//      BaseInfisicalClient ABC defines: get_secret(), set_secret(),
get_secrets(), set_secrets()
//      RealInfisicalClient handles: OAuth2 auth to Infisical, secret
fetch/store with retry
//      MockInfisicalClient uses: local JSON file for development/testing
//      Factory: get_infisical_client() → configurable via Django settings
//      NO CHANGES NEEDED to this file for the base RFP – it stays within the
//      Python vault service boundary.
//
// 3. The Python vault REST API exposes provider-specific endpoints only:
//      GET /api/v1/dfns/credentials
//      GET /api/v1/safeheron/credentials/<cosigner_uuid>/
//      POST /api/v1/creation-request/
//      There is NO generic /api/v1/credentials endpoint. The base RFP does NOT
//      need to add one – the Go client routes provider_id → correct REST
endpoint.
//
// 4. Credentials ARE currently serialised into service-level function calls
//      (dfnsService.MakePayment etc.) and the target architecture restricts
//      this to Activity-time only (WBS-BASE-17).
//
// 5. The Go client has Redis caching with fail-closed semantics (10-min
TTL).
//      The base RFP preserves this caching behaviour; Activity-time access
still
//      benefits from the cache when useCache=true.
//
// REFRIGERATION FOR WBS-BASE-08:
//      The Go polityVault.Client will be EXTENDED (not replaced) with a
//      provider-generic GetCredentials method. The existing provider-specific
//      methods remain for backward compatibility during Strangler Fig migration.
//      No new vault client is created – Activities use the existing client.
```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
----------	----------	----------------------

Table inventory	Migration and credential-source inventory	Document all GORM tables, their provider affinity, and credential content
Migration register	Migration and credential-source inventory	List all 11 gormigrate steps with their scope
Credential boundary note	Secrets remain behind Vault gateway	Confirm no direct Infisical access in WIS; note that credential objects are currently passed through service methods (to be restricted to Activity-only in WBS-BASE-08)

WBS-BASE-03: Base Contract Freeze

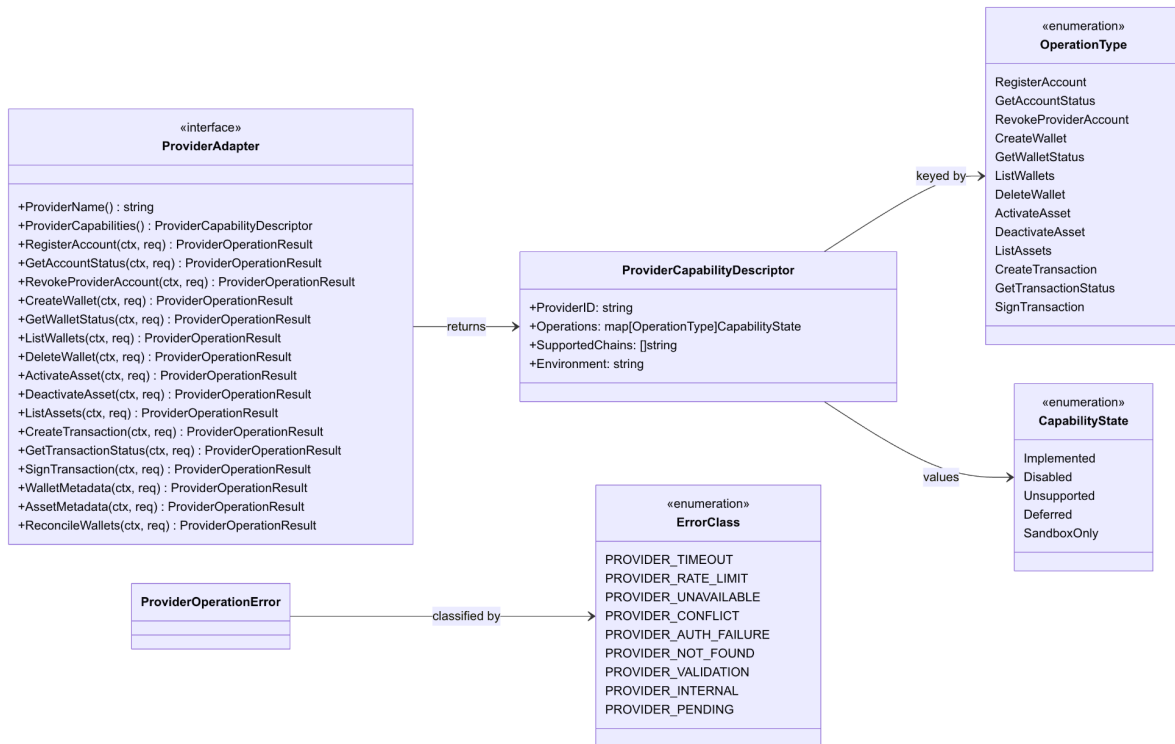
Time: 1h | MR: MR-BASE-01 | Dependencies: G0, AC-05/G2

Files to Create/Modify

File	Action	Description
<code>internal/contract/provider_adapter.go</code>	CREATE	ProviderAdapter interface definition

<code>internal/contract/provider_credentials.go</code>	CREATE	ProviderCredentials struct (references only, no secrets)
<code>internal/contract/wallet_operations.go</code>	CREATE	WalletOperationsService canonical operations
<code>internal/contract/provider_capabilities.go</code>	CREATE	CapabilityState, OperationType enums, ProviderCapabilityDescriptor
<code>internal/contract/errors.go</code>	CREATE	Canonical error taxonomy (ProviderOperationError)
<code>internal/contract/contract_test.go</code>	CREATE	Compile-proof that DFNS, Safeheron, and Fireblocks fixtures consume the same base artefact

Class Diagram — Base Contract



Key Code — ProviderAdapter Interface

```

// internal/contract/provider_adapter.go
package contract

import "context"

type ProviderAdapter interface {
    ProviderName() string
    ProviderCapabilities() ProviderCapabilityDescriptor

    // Account lifecycle
    RegisterAccount(ctx context.Context, req AccountRequest)
    ProviderOperationResult
    GetAccountStatus(ctx context.Context, req AccountRequest)
    ProviderOperationResult
    RevokeProviderAccount(ctx context.Context, req AccountRequest)
    ProviderOperationResult
    
```



```
// Wallet lifecycle
CreateWallet(ctx context.Context, req WalletRequest)
ProviderOperationResult
GetWalletStatus(ctx context.Context, req WalletRequest)
ProviderOperationResult
ListWallets(ctx context.Context, req ListWalletsRequest)
ProviderOperationResult
DeleteWallet(ctx context.Context, req WalletRequest)
ProviderOperationResult

// Asset lifecycle
ActivateAsset(ctx context.Context, req AssetRequest)
ProviderOperationResult
DeactivateAsset(ctx context.Context, req AssetRequest)
ProviderOperationResult
ListAssets(ctx context.Context, req ListAssetsRequest)
ProviderOperationResult

// Transaction scaffolding (disabled/deferred)
CreateTransaction(ctx context.Context, req TransactionRequest)
ProviderOperationResult
GetTransactionStatus(ctx context.Context, req TransactionRequest)
ProviderOperationResult
SignTransaction(ctx context.Context, req TransactionRequest)
ProviderOperationResult

// Metadata and reconciliation
WalletMetadata(ctx context.Context, req MetadataRequest)
ProviderOperationResult
AssetMetadata(ctx context.Context, req MetadataRequest)
ProviderOperationResult
ReconcileWallets(ctx context.Context, req ReconcileRequest)
ProviderOperationResult
}

type ProviderOperationResult struct {
    Success    bool
    Data       interface{}
    Error      *ProviderOperationError
    Evidence   *EvidenceReference
}

type EvidenceReference struct {
```

```

    OperationID string
    ProviderRef string
    Correlation string
    Redacted bool
}

```

Key Code — ProviderCredentials

```

// internal/contract/provider_credentials.go
package contract

// ProviderCredentials holds references and non-secret metadata only.
// Secret values are loaded at Activity execution time through Polity Vault
REST.
type ProviderCredentials struct {
    ProviderID      string
    AccountReference string
    CredentialReference string // Vault path/key, not the secret itself
    CredentialPurpose string
    RotationStatus  string
    RedactionMetadata RedactionMetadata
}

type RedactionMetadata struct {
    SchemaVersion string
    FieldsRedacted []string
    RedactedAt    string
}

```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Review contract note	DFNS, Safeheron, Fireblocks consume same artefact	Write a compile-time test that imports <code>contract</code> from three mock adapter packages to prove no local forks needed

Provider handoff assumptions	Document which methods each provider RFP must implement	Enumerate active lifecycle methods vs. disabled/deferred methods per provider
Blocked-decision log	Decisions deferred to provider RFPs	Record: Fireblocks SDK lifecycle, DFNS webhook key rotation, Safeheron callback URL — all deferred

WBS-BASE-04: ProviderAdapter Interface and Disabled Operations

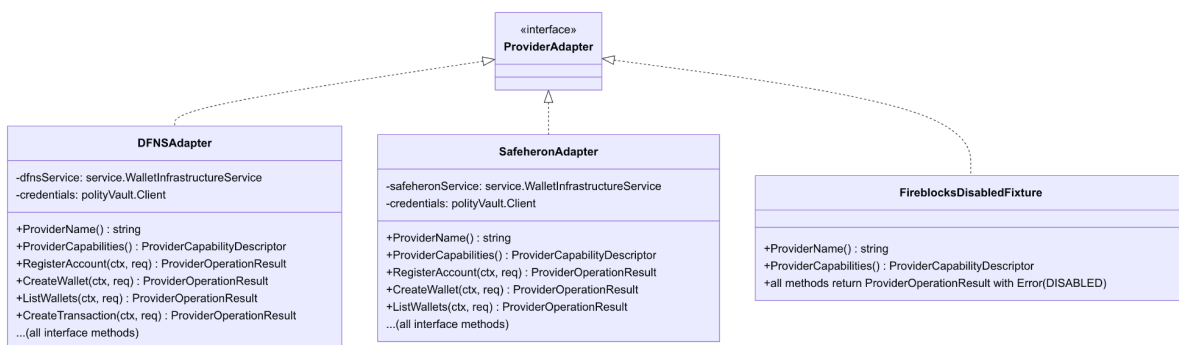
Time: 2h | MR: MR-BASE-02 | Dependencies: WBS-BASE-03, AC-05/G2, AC-07/G5

Files to Create/Modify

File	Action	Description
<code>internal/adapter/dfns/adapter.go</code>	CREATE	DFNS adapter struct implementing ProviderAdapter (initially wrapping existing service)
<code>internal/adapter/safeheron/adapter.go</code>	CREATE	Safeheron adapter struct implementing ProviderAdapter

<code>internal/adapter/fireblocks/disabled.go</code>	CR EA TE	Fireblocks disabled fixture returning DISABLED/UNSUPPORTED for all operations
<code>internal/adapter/disabled_operations_test.go</code>	CR EA TE	Unit tests proving disabled ops return canonical responses
<code>internal/contract/provider_adapter.go</code>	MO DIF Y	Refine interface based on WBS-BASE-03 freeze
<code>internal/service/interface.go</code>	MO DIF Y	Deprecate <code>WalletInfrastructureService[T]</code> payment/signing/broadcast methods in favour of adapter dispatch

Class Diagram — Adapter Pattern



Key Code — Disabled Operations


```
// internal/adapters/fireblocks/disabled.go
package fireblocks

import (
    "context"
    "github.com/Polity_MVP/wallet-infrastructure-service/internal/contract"
)

const ProviderName = "fireblocks"

type DisabledFixture struct{}

func (d *DisabledFixture) ProviderName() string { return ProviderName }

func (d *DisabledFixture) ProviderCapabilities()
contract.ProviderCapabilityDescriptor {
    ops := make(map[contract.OperationType]contract.CapabilityState)
    for _, op := range contract.AllOperationTypes() {
        ops[op] = contract.CapabilityStateDisabled
    }
    return contract.ProviderCapabilityDescriptor{
        ProviderID:      ProviderName,
        Operations:      ops,
        SupportedChains: []string{},
        Environment:     "disabled",
        DisabledReason:  "fireblocks-provider-rfp-pending",
    }
}

func (d *DisabledFixture) CreateTransaction(ctx context.Context, req
contract.TransactionRequest) contract.ProviderOperationResult {
    return contract.ProviderOperationResult{
        Success: false,
        Error: &contract.ProviderOperationError{
            ProviderID:      ProviderName,
            OperationType:   contract.CreateTransaction,
            CanonicalClass:   contract.PROVIDER_VALIDATION,
            Retryable:        false,
            BlockedByGate:    "sensitive-operation-kill-switch",
            ProviderCode:     "DISABLED",
            ProviderMessageRedacted: "transaction operations disabled for
fireblocks",
            CorrelationID:    req.CorrelationID,
        },
    },
}
```



```

    }
}

// ... each sensitive operation method (CreateTransaction, SignTransaction,
etc.)
// returns canonical disabled responses with explicit PROVIDER_VALIDATION error
class.
// Active lifecycle methods (RegisterAccount, CreateWallet, ListWallets, etc.)
// also return DISABLED since Fireblocks has no runtime adapter in base scope.

```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Unit tests for supported ops	Unit tests for supported and unsupported operations	Test each adapter method for DFNS and Safeheron that lifecycle ops return proper results; test all Fireblocks methods return DISABLED
Disabled-operation behaviour	Static proof that sensitive methods cannot reach provider runtime	Compile-time check: no import path from CreateTransaction/SignTransaction to DFNS or Safeheron SDK calls in base scope; all transaction paths return PROVIDER_VALIDATION
Compile proof	Contract type-level proof	Run <code>go build ./...</code> proving all three adapters compile against the same interface

WBS-BASE-05: Canonical Errors and Capability Metadata

Time: 2h | MR: MR-BASE-02 | Dependencies: WBS-BASE-04, AC-05/G2

Files to Create/Modify

File	Action	Description
<code>internal/contract/errors.go</code>	CREATE	Error taxonomy: retryable, non-retryable, unsupported, conflict-as-idempotent-success, deferred
<code>internal/contract/capabilities.go</code>	CREATE	Capability metadata model with fixture matrix
<code>internal/contract/errors_test.go</code>	CREATE	Table-driven tests for all error classes
<code>internal/contract/capabilities_test.go</code>	CREATE	Tests proving capability alone does not authorise execution

Key Code — Error Taxonomy

```
// internal/contract/errors.go
package contract

type ErrorClass string

const (
```

```

    PROVIDER_TIMEOUT      ErrorClass = "PROVIDER_TIMEOUT"
    PROVIDER_RATE_LIMIT   ErrorClass = "PROVIDER_RATE_LIMIT"
    PROVIDER_UNAVAILABLE  ErrorClass = "PROVIDER_UNAVAILABLE"
    PROVIDER_CONFLICT     ErrorClass = "PROVIDER_CONFLICT"
    PROVIDER_AUTH_FAILURE ErrorClass = "PROVIDER_AUTH_FAILURE"
    PROVIDER_NOT_FOUND    ErrorClass = "PROVIDER_NOT_FOUND"
    PROVIDER_VALIDATION   ErrorClass = "PROVIDER_VALIDATION"
    PROVIDER_INTERNAL     ErrorClass = "PROVIDER_INTERNAL"
    PROVIDER_PENDING      ErrorClass = "PROVIDER_PENDING"
)

type ProviderOperationError struct {
    ProviderID      string
    OperationType   OperationType
    CanonicalClass   ErrorClass
    Retryable       bool
    BlockedByGate   string
    ProviderCode    string
    ProviderMessageRedacted string
    EvidenceRef     string
    CorrelationID   string
}

func (e *ProviderOperationError) IsRetryable() bool {
    switch e.CanonicalClass {
    case PROVIDER_TIMEOUT, PROVIDER_RATE_LIMIT, PROVIDER_UNAVAILABLE,
    PROVIDER_INTERNAL:
        return true
    default:
        return false
    }
}

func (e *ProviderOperationError) IsFinal() bool {
    return !e.IsRetryable() && e.CanonicalClass != PROVIDER_PENDING
}

```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
----------	----------	----------------------

Table-driven error tests	Retryable, non-retryable, unsupported, conflict-as-idem potent-success, deferred	Create test cases for each <code>ErrorClass</code> verifying <code>IsRetryable()</code> , <code>IsFinal()</code> , conflict resolution, and deferred handling
Capability metadata model	Capability flags, supported chain metadata, display-safe provider metadata	Test that capability flags are separate from execution authorisation
Fixture matrix	DFNS, Safeheron, and Fireblocks capability fixtures	Each provider fixture lists implemented/disabled/unsupported/deferred capabilities

WBS-BASE-06: AdapterRegistry and Conformance Fixtures

Time: 2h | MR: MR-BASE-02 | Dependencies: WBS-BASE-04, WBS-BASE-05, AC-05/G2

Files to Create/Modify

File	Action	Description
<code>internal/contract/registry.go</code>	CREATE	AdapterRegistry with duplicate-provider and unknown-provider behaviour
<code>internal/contract/registry_test.go</code>	CREATE	Registry tests: deterministic selection, duplicate rejection, unknown failure, disabled fixture
<code>internal/adapter/dfns/adapter_test.go</code>	CREATE	DFNS wrapper fixture conformance test
<code>internal/adapter/safeheron/adapter_test.go</code>	CREATE	Safeheron wrapper fixture conformance test
<code>internal/adapter/fireblocks/disabled_test.go</code>	CREATE	Fireblocks disabled fixture test

Key Code — AdapterRegistry

```
// internal/contract/registry.go
package contract

import (
    "fmt"
    "sync"
)

type AdapterRegistry struct {
    mu          sync.RWMutex
    adapters    map[string]ProviderAdapter
    orderedIDs  []string
}

func NewAdapterRegistry() *AdapterRegistry {
    return &AdapterRegistry{
        adapters: make(map[string]ProviderAdapter),
    }
}

func (r *AdapterRegistry) Register(adapter ProviderAdapter) error {
    r.mu.Lock()
    defer r.mu.Unlock()
    name := adapter.ProviderName()
    if _, exists := r.adapters[name]; exists {
        return fmt.Errorf("duplicate provider registration: %s", name)
    }
    r.adapters[name] = adapter
    r.orderedIDs = append(r.orderedIDs, name)
    return nil
}

func (r *AdapterRegistry) Get(providerID string) (ProviderAdapter, error) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    adapter, ok := r.adapters[providerID]
    if !ok {
        return nil, &ProviderOperationError{
            ProviderID:    providerID,
            CanonicalClass: PROVIDER_NOT_FOUND,
            Retryable:     false,
        }
    }
    return adapter, nil
}
```

```

        ProviderCode: "UNKNOWN_PROVIDER",
        ProviderMessageRedacted: fmt.Sprintf("provider %s is not
registered", providerID),
    }
}
return adapter, nil
}

func (r *AdapterRegistry) MustGet(providerID string) ProviderAdapter {
    adapter, err := r.Get(providerID)
    if err != nil {
        panic(err)
    }
    return adapter
}

func (r *AdapterRegistry) ListProviders() []ProviderCapabilityDescriptor {
    r.mu.RLock()
    defer r.mu.RUnlock()
    var result []ProviderCapabilityDescriptor
    for _, id := range r.orderedIDs {
        result = append(result, r.adapters[id].ProviderCapabilities())
    }
    return result
}

```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Registry duplicate rejection test	Deterministic provider selection, duplicate rejection	Register DFNS adapter twice → second call returns error

Unknown-provider failure test	Unknown-provider failure	<code>Get("nonexistent")</code> returns <code>PROVIDER_NOT_FOUND</code> error
Disabled fixture behaviour test	Disabled Fireblocks fixture behaviour	All Fireblocks methods return <code>DISABLED</code> with <code>BlockedByGate</code>
Capability matrix test	Provider capability fixtures	Verify DFNS has <code>Implemented</code> for lifecycle ops, Fireblocks has <code>Disabled</code> for all

WBS-BASE-07: Feature Gates and Sensitive-Operation Kill Switch

Time: 2h | MR: MR-BASE-02 | Dependencies: WBS-BASE-06, AC-07/G5, AC-09/G6

Files to Create/Modify

File	Action	Description
<code>flagger/flagger.go</code>	MODIFY	Add global wallet, provider, operation, environment, and

		sensitive-operation kill switch flags
<code>config/config.go</code>	MODIFY	Add gate configuration fields
<code>internal/contract/gates.go</code>	CREATE	Canonical gate precedence matrix and kill switch logic
<code>internal/contract/gates_test.go</code>	CREATE	Matrix tests proving all gates must pass before workflow start

Key Code — Feature Gates

```
// internal/contract/gates.go
package contract

type GateResult struct {
    Passed      bool
    GateName    string
    BlockedBy   string
    Evidence    string
}

type OperationContext struct {
    ProviderID      string
    OperationType   OperationType
    UserID          string
    Environment     string
    IdempotencyKey  string
    CorrelationID   string
}

type GateChain struct {
    globalWalletEnabled bool
    providerEnabled      map[string]bool
    operationEnabled     map[OperationType]bool
}
```

```

environmentGate      map[string]bool
sensitiveOpKillSwitch bool
registry             *AdapterRegistry
}

// EvaluateGates returns the FIRST failing gate or success.
// Gate precedence: global → provider → operation → environment → kill-switch →
// capability → idempotency
func (gc *GateChain) EvaluateGates(ctx OperationContext) GateResult {
    // Gate 1: Global wallet enablement
    if !gc.globalWalletEnabled {
        return GateResult{Passed: false, GateName: "global_wallet", BlockedBy:
"wallet_globally_disabled"}
    }
    // Gate 2: Provider enablement
    if !gc.providerEnabled[ctx.ProviderID] {
        return GateResult{Passed: false, GateName: "provider_enablement",
BlockedBy: "provider_disabled"}
    }
    // Gate 3: Operation enablement
    if !gc.operationEnabled[ctx.OperationType] {
        return GateResult{Passed: false, GateName: "operation_enablement",
BlockedBy: "operation_disabled"}
    }
    // Gate 4: Environment/segment gate
    if !gc.environmentGate[ctx.Environment] {
        return GateResult{Passed: false, GateName: "environment_gate",
BlockedBy: "environment_not_approved"}
    }
    // Gate 5: Sensitive-operation kill switch
    if gc.sensitiveOpKillSwitch && isSensitiveOperation(ctx.OperationType) {
        return GateResult{Passed: false, GateName: "sensitive_kill_switch",
BlockedBy: "sensitive_operation_blocked"}
    }
    // Gate 6: Provider capability
    adapter, err := gc.registry.Get(ctx.ProviderID)
    if err != nil {
        return GateResult{Passed: false, GateName: "provider_capability",
BlockedBy: "provider_not_found"}
    }
    caps := adapter.ProviderCapabilities()
    state, ok := caps.Operations[ctx.OperationType]
    if !ok || state != CapabilityStateImplemented {

```



```

        return GateResult{Passed: false, GateName: "provider_capability",
BlockedBy: string(state)}
    }
    // Gate 7: Idempotency (handled in WBS-BASE-12)
    return GateResult{Passed: true, GateName: "all_gates", BlockedBy: ""}
}

func isSensitiveOperation(op OperationType) bool {
    switch op {
    case CreateTransaction, GetTransactionStatus, SignTransaction:
        return true
    default:
        return false
    }
}

```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Gate matrix test	Every required gate must pass before workflow start	Test all 7 gates individually, test that failing any single gate prevents dispatch
Kill-switch fail-closed test	Kill-switch failure is fail-closed and auditable	Set <code>sensitiveOpKillSwitch = true</code> → all sensitive ops return blocked; set <code>false</code> → only capability-gated ops still blocked
Rollback note	Feature flag rollback	Document how to revert each flag to restore legacy-safe behavior

WBS-BASE-08: Vault Gateway

Request Contract

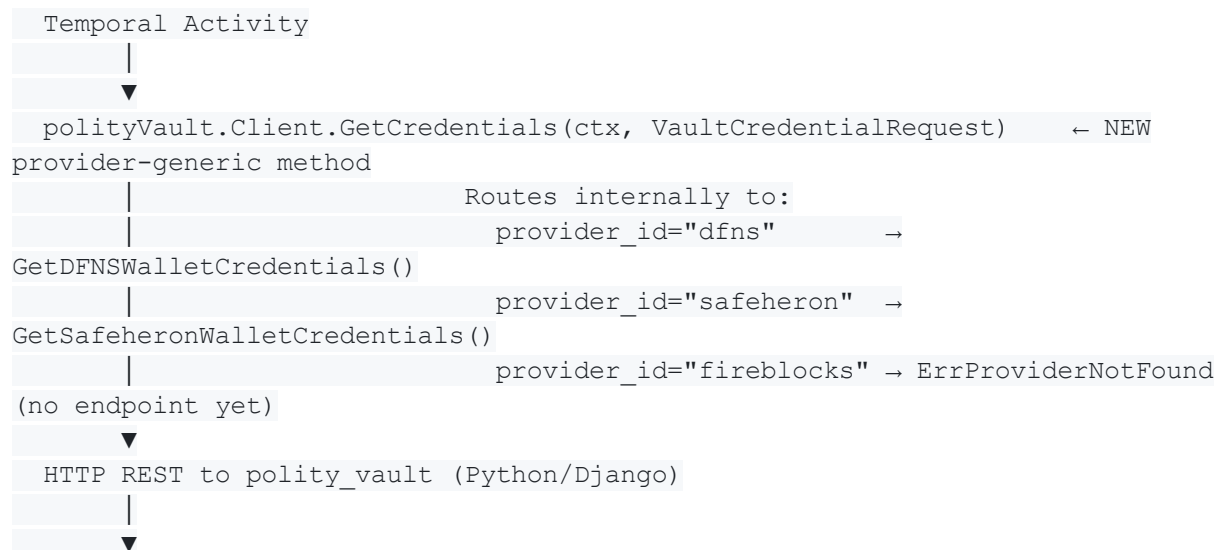
Time: 7h | MR: MR-BASE-02 | Dependencies: WBS-BASE-04, AC-03/G3-G4

Architecture Context — Existing Vault Integration

The credential boundary already has a well-structured three-tier architecture. No new vault client is needed. The existing `polityVault.Client` in Go calls the Python `polity_vault` REST service, which in turn uses `vault_backend/client.py` (`RealInfisicalClient`) to fetch secrets from Infisical. The RFP requirement "WIS must not call Infisical directly" is already satisfied — WIS never calls Infisical.

What changes in this WBS is: (1) extending the Go client with a provider-generic method, (2) defining the Activity-time contract that ensures credentials never leak into workflow state, and (3) confirming the Python vault service needs no changes for the base RFP.

CREDENTIAL FLOW (current and target):



`vault_backend/client.py` → `RealInfisicalClient` → Infisical API

Credentials are:

- ✓ Loaded only at Activity execution time (never in workflow input/state)
- ✓ Never serialised into workflow history, logs, or fixtures
- ✓ Cached in Redis (fail-closed, 10-min TTL) – preserved behaviour
- ✓ Not stored in ordinary wallet rows or provider metadata

Files to Create/Modify

File	Action	Description
<code>internal/contract/vault_gateway.go</code>	CREATE	Activity-time credential request/response contract (references only, no secrets)
<code>internal/contract/vault_gateway_test.go</code>	CREATE	No-secret tests, credential failure matrix
<code>providers/polityVault/client.go</code>	MODIFY	Add provider-generic <code>GetCredentials(ctx, VaultCredentialRequest)</code> method that routes to existing provider-specific methods
<code>providers/polityVault/client_test.go</code>	MODIFY	Add tests for the new <code>GetCredentials</code> method covering all routing paths and failure modes
<code>providers/polityVault/mock.go</code>	MODIFY	Add <code>GetCredentials</code> to mock client

Files NOT Changed (and why)

File	Reason
<code>polity_vault/vault_backend/client.py</code>	The Python Infisical client is internal to the vault service. WIS calls only the REST API. The <code>BaseInfisicalClient</code> , <code>RealInfisicalClient</code> , and <code>MockInfisicalClient</code> stay unchanged — they are the vault service's internal dependency.
<code>polity_vault/wallet_credentials/views.py</code>	The vault REST endpoints (<code>/api/v1/dfns/credentials</code> , <code>/api/v1/safeheron/credentials/<uuid>/</code> , <code>/api/v1/creation-request/</code>) remain provider-specific. The Go client routes <code>provider_id</code> → correct endpoint internally. A generic <code>/api/v1/credentials</code> endpoint is NOT needed for the base RFP.
<code>polity_vault/wallet_credentials/models.py</code>	The <code>DFNSWalletInfo</code> and <code>SafeheronWalletInfo</code> models and their <code>WalletCreationRequest</code> lifecycle remain unchanged. Fireblocks credential models will be added in the Fireblocks provider RFP.

Key Code — Vault Gateway Contract (internal/contract/vault_gateway.go)

```
// internal/contract/vault_gateway.go
package contract
```

```
// VaultCredentialRequest is the Activity-time request to the Polity Vault REST
gateway.
// It contains only references, never secret values.
```



```
// This is the contract type that Activities use to request credentials.
// The actual credential fetching is delegated to polityVault.Client.
type VaultCredentialRequest struct {
    ProviderID      string
    AccountReference string // e.g. cosigner_uuid for Safeheron
    CredentialReference string // Optional vault path override
    CredentialPurpose string // e.g. "wallet_creation", "asset_activation"
    CorrelationID   string
    UseCache        bool    // Whether to use Redis credential caching
}

// VaultCredentialResponse contains non-secret metadata about the loaded
// credentials.
// The actual credential values (SafeheronCredentials, DFNSCredentials) are
// provider-specific objects that live ONLY within Activity scope and are never
// serialised into workflow inputs, history, logs, or fixtures.
type VaultCredentialResponse struct {
    ProviderID      string
    AccountReference string
    LoadedAt        string
    ExpiresAt       string // From cache TTL or vault response
    CacheHit        bool    // Whether credentials came from Redis cache
    // NOTE: Actual credential values are NOT in this struct.
    // They are held by the Activity in provider-specific scope
    // (vault.SafeheronCredentials or vault.DFNSCredentials)
    // and passed directly to the ProviderAdapter – never to workflow state.
}
```

Key Code — Extended polityVault.Client (providers/polityVault/client.go)

```
// providers/polityVault/client.go – EXTENDED (existing methods preserved for
// Strangler Fig)

// The Client interface is extended with GetCredentials.
// Existing GetSafeheronWalletCredentials, GetDFNSWalletCredentials, and
// CreateWalletRequest remain unchanged for backward compatibility.

type Client interface {
    // GetCredentials is the provider-generic credential request method.
    // It routes to the correct provider-specific method based on ProviderID.
```



```

// This is the method Activities should use in the target architecture.
GetCredentials(ctx context.Context, req contract.VaultCredentialRequest)
(interface{}, int, error)

// Existing provider-specific methods preserved for Strangler Fig
migration.
GetSafeheronWalletCredentials(token, cosigner_uuid string, options
*DownloadOption) (*vault.SafeheronCredentials, int, error)
GetDFNSWalletCredentials(token string, options *DownloadOption)
(*vault.DFNSCredentials, int, error)
CreateWalletRequest(vault.WalletCreationRequest, string) error
DefaultOptions() *DownloadOption
}

// GetCredentials routes to the correct provider-specific credential method.
// It returns the raw provider-specific credential object (SafeheronCredentials
// or DFNSCredentials) which the caller (an Activity) uses directly with the
// ProviderAdapter. The credential values never enter workflow state.
func (c *client) GetCredentials(ctx context.Context, req
contract.VaultCredentialRequest) (interface{}, int, error) {
    switch req.ProviderID {
    case "dfns":
        // DFNS credentials use a service-level JWT token (not user-scoped).
        // The token is sourced from the Activity context
        (VaultCredentialRequest.CorrelationID
        // maps to the auth context for the operation).
        opts := &DownloadOption{useCache: req.UseCache}
        creds, code, err := c.GetDFNSWalletCredentials(/* token from context
        */, opts)
        if err != nil {
            return nil, code, mapVaultError(req.ProviderID, err)
        }
        return creds, code, nil

    case "safeheron":
        // Safeheron credentials require a cosigner_uuid as account reference.
        opts := &DownloadOption{useCache: req.UseCache}
        creds, code, err := c.GetSafeheronWalletCredentials(/* token */,
        req.AccountReference, opts)
        if err != nil {
            return nil, code, mapVaultError(req.ProviderID, err)
        }
        return creds, code, nil
    }
}

```

```

    case "fireblocks":
        // Fireblocks has no vault endpoint yet – this is a disabled fixture.
        // Returns PROVIDER_NOT_FOUND with no vault call.
        return nil, 0, &contract.ProviderOperationError{
            ProviderID:          req.ProviderID,
            OperationType:       contract.RegisterAccount,
            CanonicalClass:      contract.PROVIDER_NOT_FOUND,
            Retryable:           false,
            ProviderCode:        "NO_CREDENTIAL_ENDPOINT",
            ProviderMessageRedacted: "fireblocks credential endpoint not
available in base RFP",
            CorrelationID:      req.CorrelationID,
        }

    default:
        return nil, 0, &contract.ProviderOperationError{
            ProviderID:          req.ProviderID,
            CanonicalClass:      contract.PROVIDER_NOT_FOUND,
            Retryable:           false,
            ProviderCode:        "UNKNOWN_PROVIDER",
            ProviderMessageRedacted: fmt.Sprintf("provider %s has no credential
endpoint", req.ProviderID),
            CorrelationID:      req.CorrelationID,
        }
    }
}

// mapVaultError converts polityVault sentinel errors to canonical
ProviderOperationErrors.
func mapVaultError(providerID string, err error)
*contract.ProviderOperationError {
    switch {
    case errors.Is(err, ErrCacheUnavailable):
        return &contract.ProviderOperationError{
            ProviderID:          providerID,
            CanonicalClass:      contract.PROVIDER_UNAVAILABLE,
            Retryable:           true, // Cache unavailable is retryable
per RFP §20.1.5
            ProviderCode:        "VAULT_CACHE_UNAVAILABLE",
            ProviderMessageRedacted: "credential cache temporarily
unavailable",
        }
    case errors.Is(err, ErrUnauthorized):
        return &contract.ProviderOperationError{

```

```

        ProviderID:          providerID,
        CanonicalClass:      contract.PROVIDER_AUTH_FAILURE,
        Retryable:           false, // Auth failure is non-retryable
per RFP
        ProviderCode:        "VAULT_AUTH_FAILURE",
        ProviderMessageRedacted: "credential authorization failed",
    }
    case errors.Is(err, ErrNotFound):
        return &contract.ProviderOperationError{
            ProviderID:          providerID,
            CanonicalClass:      contract.PROVIDER_NOT_FOUND,
            Retryable:           false,
            ProviderCode:        "CREDENTIAL_NOT_FOUND",
            ProviderMessageRedacted: "credential reference not found in vault",
        }
    default:
        return &contract.ProviderOperationError{
            ProviderID:          providerID,
            CanonicalClass:      contract.PROVIDER_INTERNAL,
            Retryable:           true, // Default to retryable for unknown
vault errors
            ProviderCode:        "VAULT_INTERNAL_ERROR",
            ProviderMessageRedacted: redactSecrets(err.Error()),
        }
    }
}

```

Rationale: Why No Separate Vault Client Is Needed

A dedicated vault client for Activities (e.g., `internal/temporal/vault_client.go`) would be redundant and is intentionally omitted. The reasons are:

1. The existing `polityVault.Client` already implements the Polity Vault REST gateway boundary. It handles Redis caching (fail-closed), key formatting, error mapping, and HTTP communication with the Python vault service. Creating a parallel client would duplicate all of this logic.
2. The Python `vault_backend/client.py` (`RealInfisicalClient`) is the Infisical integration point. It stays within the Python vault service boundary. The RFP explicitly states "WIS must not call Infisical directly" — the Go client already satisfies this by calling only the REST API.

3. The approach of extending `polityVault.Client` with a `GetCredentials` method follows the RFP's Strangler Fig pattern: existing provider-specific methods remain for backward compatibility while the new generic method becomes the Activity-time entry point.
4. Activities do not need their own vault client. They call `polityVault.Client.GetCredentials()` which internally routes to the correct provider-specific credential method. The credential values stay in Activity scope and never enter workflow state.
5. The Redis caching in the existing client is preserved. Activities can set `UseCache: true` in `VaultCredentialRequest` to benefit from the fail-closed caching, or `UseCache: false` for immediate fresh credentials.

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
No-secret tests	Credentials requested only through Vault gateway at Activity execution time	Test <code>GetCredentials</code> for each provider; verify <code>VaultCredentialResponse</code> contains no secret values; verify workflow inputs never contain credentials

Cred entia l failur e matr ix	Missin g refere nce, stale, expire d, unavai lable	Test <code>GetCredentials</code> returns canonical errors for: unknown provider, fireblocks (disabled), <code>ErrCacheUnavailable</code> → <code>PROVIDER_UNAVAILABLE</code> (retryable), <code>ErrUnauthorized</code> → <code>PROVIDER_AUTH_FAILURE</code> (non-retryable), <code>ErrNotFound</code> → <code>PROVIDER_NOT_FOUND</code> (non-retryable)
Cod e scan	No direct Infisica l access from WIS	<code>grep -r "infisical" internal/</code> returns zero; <code>grep -r "infisical" providers/</code> returns zero (the only Infisical calls are in the Python vault service)
Vaul t bou ndar y confi rmat ion	Crede ntials are fetche d throug h polityV ault.Cli ent REST → Python vault →	Verify no code path in WIS calls Infisical directly; the Python <code>vault_backend/client.py</code> is the sole Infisical integration point

	Infisical	
Redaction rules	Secret hygiene tests	Verify <code>mapVaultError</code> redacts all secret material from error messages before they reach workflow state
Python vault service unchanged	Base RFP does not modify <code>polity_vault</code>	Confirm <code>polity_vault/</code> files are not modified in MR-BASE-02; only <code>providers/polityVault/</code> and <code>internal/contract/</code> are changed

WBS-BASE-09: Provider-Neutral Metadata Migrations

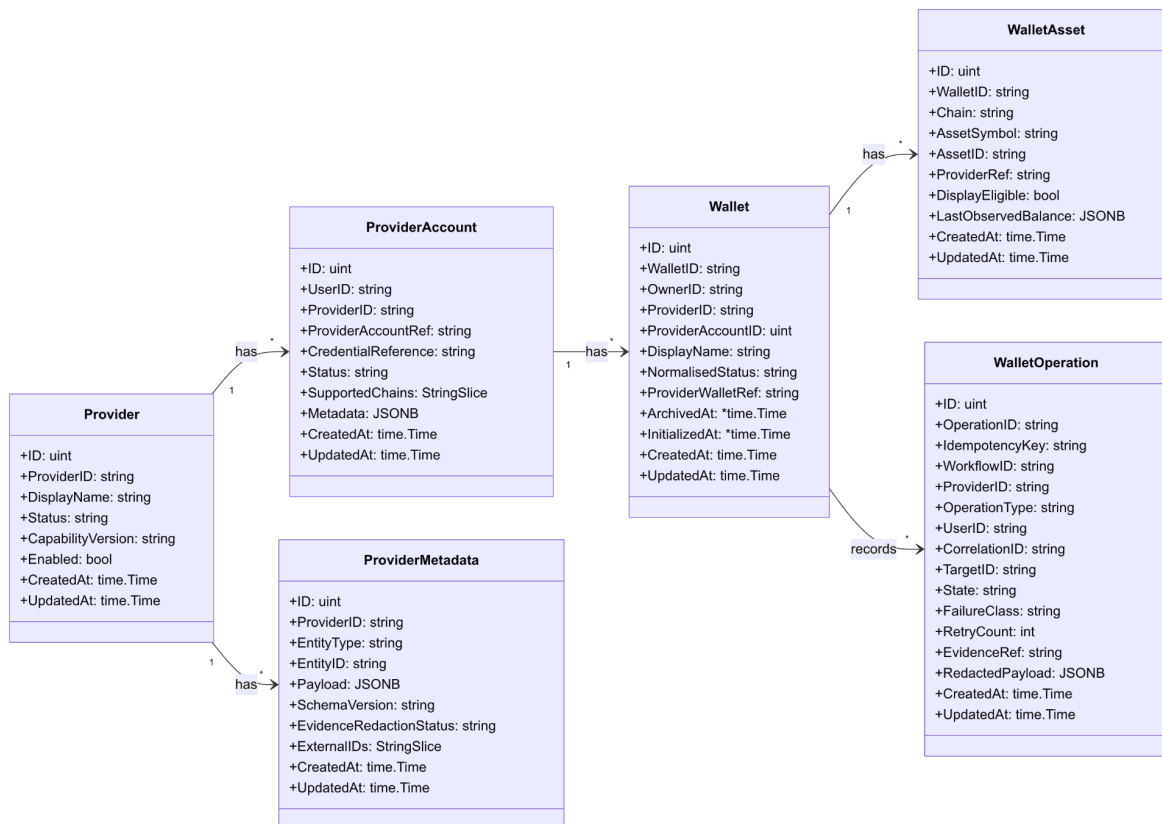
Time: 3h | MR: MR-BASE-03 | Dependencies: WBS-BASE-02, AC-04/G6

Files to Create/Modify

File	Action	Description
------	--------	-------------

<code>internal/storage/migrations/migrations.go</code>	MODIFY	Add new gormigrate steps for provider-neutral tables
<code>internal/models/provider/provider.go</code>	CREATE	Provider model
<code>internal/models/provider/account.go</code>	CREATE	ProviderAccount model
<code>internal/models/wallet/wallet.go</code>	CREATE	Provider-neutral Wallet model (new, separate from <code>polity.Wallet</code>)
<code>internal/models/wallet/asset.go</code>	CREATE	WalletAsset model
<code>internal/models/operation/operation.go</code>	CREATE	WalletOperation model (WALLET_OPERATIONS)
<code>internal/models/provider/metadata.go</code>	CREATE	ProviderMetadata model
<code>internal/contract/provider.go</code>	CREATE	Provider table constants and seed data

Class Diagram — Persistence Model



Key Code — Migration Steps

```

// internal/storage/migrations/migrations.go - ADD new steps after 000000000011
// New steps: 000000000012 through 000000000017

// Step 000000000012: providers table
// Step 000000000013: provider_accounts table
// Step 000000000014: wallets (provider-neutral, with read-compatibility view
// for legacy)
// Step 000000000015: wallet_assets table
// Step 000000000016: wallet_operations table (WALLET_OPERATIONS)
// Step 000000000017: provider_metadata table

// Each step includes:
// - Migration up: CREATE TABLE with constraints and indexes

```

```
// - Migration down: DROP TABLE (or DROP VIEW for compatibility views)
// - Uniqueness: provider_id + operation_type + idempotency_key on
wallet_operations
// - JSONB GIN index on provider_metadata.payload
// - No credential material stored in any table
```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Migration up/down logs	Up/down logs, schema diff	Run <code>gormigrate.Migrate()</code> and <code>gormigrate.Rollback()</code> for each step
Schema notes	Field-level classification, retention/redaction notes	Document each table's classification (secret-free, display-safe, provider-identifying)
Storage tests	Provider-neutral rows are reversible and contain no credential material	Test INSERT/SELECT/DROP for each new table; verify no secret fields exist

WBS-BASE-10: Seed and Legacy-Read Compatibility

Time: 2h | MR: MR-BASE-03 | Dependencies: WBS-BASE-09, AC-10/G6-G7

Files to Create/Modify

File	Action	Description
<code>internal/storage/seed.go</code>	CREATE	Seed data for approved provider IDs and release chain set
<code>internal/storage/compatibility.go</code>	CREATE	Legacy read-compatibility layer for DFNS/Safeheron rows
<code>internal/storage/seed_test.go</code>	CREATE	Seed and read-compatibility tests
<code>internal/contract/chains.go</code>	CREATE	Approved release chain set constants

Key Code — Seed Data

```
// internal/storage/seed.go
package storage

type ProviderSeed struct {
```



```

    ProviderID      string
    DisplayName     string
    Status          string
    CapabilityVersion string
    Enabled         bool
}

var ApprovedProviders = []ProviderSeed{
    {ProviderID: "dfns", DisplayName: "DFNS", Status: "active",
    CapabilityVersion: "1.0", Enabled: true},
    {ProviderID: "safeheron", DisplayName: "Safeheron", Status: "active",
    CapabilityVersion: "1.0", Enabled: true},
    {ProviderID: "fireblocks", DisplayName: "Fireblocks", Status: "disabled",
    CapabilityVersion: "0.0", Enabled: false},
}

type ChainSeed struct {
    ChainID      string
    Name         string
    IsTestnet    bool
    Approved     bool
}

var ApprovedReleaseChainSet = []ChainSeed{
    // Populated from governance-controlled chain list at engagement start
    // Non-baseline networks require governance change control
}

```

Key Code — Legacy Read Compatibility

```

// internal/storage/compatibility.go
package storage

// LegacyWalletRead reads existing DFNS/Safeheron wallet rows during Strangler
Fig coexistence.
// This layer maps provider-specific rows to the new provider-neutral Wallet
model.
func (s *WalletInfrastructureStorage) ReadLegacyWallets(userID string,
providerType string) ([]LegacyWalletView, error) {
    switch providerType {
    case "dfns":
        return s.readDFNSWallets(userID)
    }
}

```

```

case "safeheron":
    return s.readSafeheronWallets(userID)
default:
    return nil, fmt.Errorf("unsupported provider: %s", providerType)
}

```

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Seed row tests	Approved release chain set metadata works	Test that approved chains are queryable and non-baseline chains are rejected/disabled
Read-compatibility tests	Legacy rows still read correctly	Run read queries against existing DFNS/Safeheron data through compatibility layer
Chain-boundary note	Non-baseline networks rejected or disabled	Test that <code>ApprovedReleaseChainSet</code> rejects unapproved chains

WBS-BASE-11: WALLET_OPERATIONS Ledger Schema

Time: 3h | MR: MR-BASE-03 | Dependencies: WBS-BASE-09, AC-04/G6

Files to Create/Modify

File	Action	Description
<code>internal/models/operation/operation.go</code>	MODIFY	Full WalletOperation model with all required fields
<code>internal/storage/migrations/migrations.go</code>	MODIFY	Add WALLET_OPERATIONS migration with uniqueness constraints
<code>internal/repository/operation_repository.go</code>	CREATE	Repository methods for WALLET_OPERATIONS

Key Code — WalletOperation Model

```
// internal/models/operation/operation.go
package operation

import (
    "time"
    "gorm.io/gorm"
)

type OperationState string

const (
```

```

    StatePending           OperationState = "PENDING"
    StateProvisioning       OperationState = "PROVISIONING"
    StateCompleted          OperationState = "COMPLETED"
    StateFailed             OperationState = "FAILED"
    StateConflictIdempotent OperationState = "CONFLICT_IDEMPOTENT"
    StateCompensationStarted OperationState = "COMPENSATION_STARTED"
    StateCompensationFailed OperationState = "COMPENSATION_FAILED"
    StateRetrying           OperationState = "RETRYING"
    StateBlocked            OperationState = "BLOCKED"
    StateCancelled          OperationState = "CANCELLED"
    StateDeferred           OperationState = "DEFERRED"
)

type WalletOperation struct {
    gorm.Model
    OperationID string `gorm:"uniqueIndex;not null"
    json:"operation_id"`
    IdempotencyKey string `gorm:"uniqueIndex:idx_idempotency;not null"
    json:"idempotency_key"`
    WorkflowID string `json:"workflow_id"`
    ProviderID string `gorm:"uniqueIndex:idx_idempotency;not null"
    json:"provider_id"`
    OperationType string `gorm:"uniqueIndex:idx_idempotency;not null"
    json:"operation_type"`
    UserID string `json:"user_id"`
    CorrelationID string `json:"correlation_id"`
    TargetID string `json:"target_id"`
    State OperationState `gorm:"index;not null" json:"state"`
    FailureClass string `json:"failure_class"`
    RetryCount int `json:"retry_count"`
    EvidenceRef string `json:"evidence_ref"`
    RedactedPayload gorm/datatypes.JSON `json:"redacted_payload"`
    Compensations []Compensation `gorm:"foreignKey=OperationID"
    json:"compensations,omitempty"`
    CreatedAt time.Time
    UpdatedAt time.Time
}

```

Idempotency Key Responsibility and Derivation

The idempotency key is caller-provided. The API consumer (frontend, internal service, or Temporal workflow) is responsible for generating and supplying the `IdempotencyKey`

value when initiating a wallet operation. This follows the same pattern used by idempotent payment APIs (e.g., Stripe): the caller generates a client-side unique identifier (typically a UUID v4) and can safely retry the same request with the same key to receive a deterministic result without duplicate side effects.

Key derivation and uniqueness rules:

Layer	Responsibility	Rule
API caller	Generates the <code>IdempotencyKey</code>	Client-generated UUID v4 or equivalent unique identifier, supplied in the request body
WALLET_OPERATION	Enforces uniqueness	Composite unique constraint (<code>provider_id</code> , <code>operation_type</code> , <code>idempotency_key</code>) — duplicate insertions resolve as <code>CONFLICT_IDEMPOTENT_SUCCESS</code>
Temporal workflow	Derives workflow ID deterministically	<code>wallet-{provider_id}-{operation_type}-{idempotency_key}</code> — ensures replay and conflict behaviour is predictable
Repository	CreateOrFind semantics	If a record with the same (<code>provider_id</code> , <code>operation_type</code> , <code>idempotency_key</code>) already exists and is in a terminal state, return the existing record without creating a duplicate or triggering a provider side effect

Example request flow:

1. Frontend calls `POST /api/wallets/v1/` with:


```
{ "idempotency_key": "550e8400-e29b-41d4-a716-446655440000",
  "provider_id": "dfns", "operation_type": "provision_wallet", ... }
```

2. WalletOperationsService checks gate chain (global, provider, operation, environment, kill-switch, capability, idempotency)

3. Idempotency gate: repository.CreateOrFind(provider_id="dfns", operation_type="provision_wallet", idempotency_key="550e8400-e29b-41d4-a716-446655440000")

- If found and terminal → return existing result (CONFLICT_IDEMPOTENT_SUCCESS)
- If not found → create WALLET_OPERATIONS row in PENDING state, then start Temporal workflow

4. Temporal workflow ID = "wallet-dfns-provision_wallet-550e8400-e29b-41d4-a716-446655440000"

- Deterministic: same inputs always produce the same workflow ID
- Replay-safe: if the workflow already exists, Temporal returns the existing run

Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Migration tests with uniqueness	Uniqueness and deterministic replay	Test that duplicate idempotency_key + provider_id + operation_type raises constraint violation
Repository tests	No sensitive payload persistence	Verify WalletOperation contains no credential material or signing data
Redacted evidence storage	Redacted evidence references	Verify `RedactedPayload` field uses JSONB with classification notes, no secrets

WBS-BASE-12: Idempotency Repository Behaviour

Time: 3h | **MR**: MR-BASE-03 | **Dependencies**: WBS-BASE-11, AC-04/G6

Files to Create/Modify

File	Action	Description

```
| `internal/repository/operation_repository.go` | **MODIFY** | Add
CreateOrFind, conflict-as-idempotent-success logic |
| `internal/repository/operation_repository_test.go` | **CREATE** | Concurrency
tests and replay tests |
```

Key Code – Idempotency Repository

```
```go
// internal/repository/operation_repository.go
package repository

type WalletOperationRepository interface {
 // CreateOrFind creates a WALLET_OPERATIONS record before external effects.
 // If a record with the same (provider_id, operation_type, idempotency_key)
exists,
 // it returns the existing record as CONFLICT_IDEMPOTENT_SUCCESS.
 CreateOrFind(ctx context.Context, op *operation.WalletOperation)
(*operation.WalletOperation, error)
 UpdateState(ctx context.Context, operationID string, state
operation.OperationalState, evidence string) error
 GetByIdempotencyKey(ctx context.Context, providerID, opType, idempotencyKey
string) (*operation.WalletOperation, error)
}

func (r *GormOperationRepository) CreateOrFind(ctx context.Context, op
*operation.WalletOperation) (*operation.WalletOperation, error) {
 result := r.db.WithContext(ctx).Where(
 "provider_id = ? AND operation_type = ? AND idempotency_key = ?",
 op.ProviderID, op.OperationType, op.IdempotencyKey,
).First(op)

 if result.Error == nil {
 // Idempotency key exists → return CONFLICT_IDEMPOTENT_SUCCESS
 return op, nil
 }

 if !errors.Is(result.Error, gorm.ErrRecordNotFound) {
 return nil, result.Error
 }

 // Create new operation record (BEFORE external side effects)
 op.State = operation.StatePending
 if err := r.db.WithContext(ctx).Create(op).Error; err != nil {
 // Handle concurrent insert race condition

```

```

 if strings.Contains(err.Error(), "duplicate key") {
 return r.GetByIdempotencyKey(ctx, op.ProviderID, op.OperationType,
 op.IdempotencyKey)
 }
 return nil, err
 }
 return op, nil
}

```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Concurrency tests	Duplicate requests do not create duplicate provider side effects	Simulate concurrent CreateOrFind calls with same idempotency key → only one record created
Replay tests	Conflict-as-idempotent-success	Replay same operation → returns existing record, no second provider call
Idempotency key caller responsibility	Idempotency key is caller-provided	Verify the API handler accepts idempotency_key from the request body, validates it is non-empty and well-formed (UUID v4), and passes it through to CreateOrFind; verify that the same idempotency_key submitted twice returns the same WALLET_OPERATIONS record

Operation correlation	Create WALLET_OPERATIONS before provider side effects	Verify in workflow that operation record exists before any adapter call
-----------------------	-------------------------------------------------------	-------------------------------------------------------------------------

## WBS-BASE-13: Persistence Rollback Package

Time: 1h | MR: MR-BASE-03 | Dependencies: WBS-BASE-09..12, AC-04/G6, AC-10/G6-G7

### Files to Create/Modify

File	Action	Description
<code>internal/storage/rollback.go</code>	CREATE	Rollback evidence, backup assumptions, residual risk notes
<code>internal/storage/rollback_test.go</code>	CREATE	Rollback replay tests
<code>docs/rollback-notes.md</code>	CREATE	Rollback procedure documentation per migration

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Up/down evidence	Reviewer can replay up/down evidence	Run each migration forward and backward, capture logs
Data-risk blockers	Identify data-risk blockers before provider-specific work	Document: Cannot drop legacy tables until compatibility bridge is accepted
Migration evidence index	Index of all migrations with rollback paths	Table of migration IDs, up/down results, residual risks

## WBS-BASE-14: Direct Temporal Client Boundary

Time: 2h | MR: MR-BASE-04 | Dependencies: WBS-BASE-04, WBS-BASE-11, AC-01/G1, AC-02/G1-G3, AC-08/G3

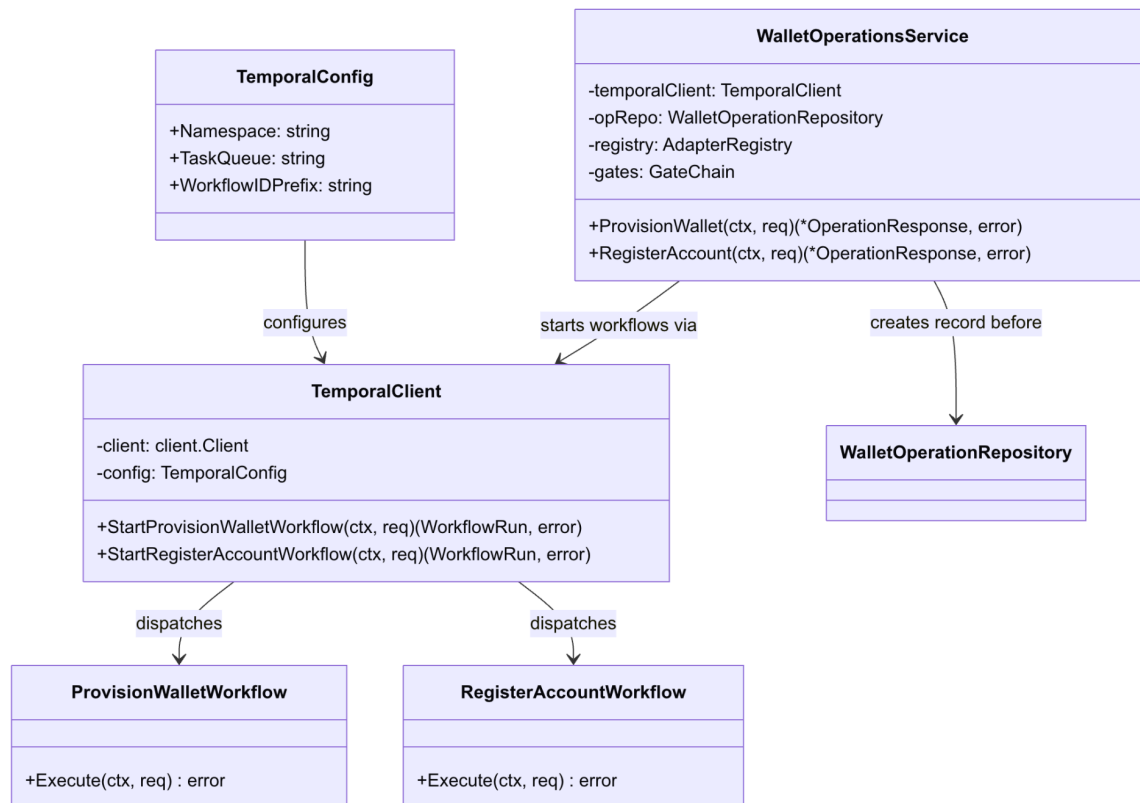
### Files to Create/Modify

File	Action	Description
------	--------	-------------



<code>internal/temporal/client.go</code>	CREATE	Temporal Go SDK client wrapper with <code>client.ExecuteWorkflow()</code>
<code>internal/temporal/config.go</code>	CREATE	Namespace, task queue, workflow ID conventions
<code>internal/temporal/workflows.go</code>	CREATE	ProvisionWalletWorkflow and RegisterAccountWorkflow definitions
<code>internal/temporal/client_test.go</code>	CREATE	Workflow-start tests, ID correlation, no-Nexus proof

## Class Diagram — Temporal Boundary



## Key Code — Temporal Client

```
// internal/temporal/client.go
package temporal

import (
 "context"
 "fmt"
 "go.temporal.io/sdk/client"
)

const (
 Namespace = "polity-wallet"
 TaskQueue = "wallet-operations"
)
```

```

type WalletTemporalClient struct {
 client client.Client
}

func NewWalletTemporalClient(c client.Client) *WalletTemporalClient {
 return &WalletTemporalClient{client: c}
}

func deterministicWorkflowID(providerID, opType, idempotencyKey string) string {
 return fmt.Sprintf("wallet-%s-%s-%s", providerID, opType, idempotencyKey)
}

func (tc *WalletTemporalClient) StartProvisionWalletWorkflow(
 ctx context.Context, req ProvisionWalletRequest,
) (client.WorkflowRun, error) {
 opts := client.StartWorkflowOptions{
 ID: deterministicWorkflowID(req.ProviderID,
"provision_wallet", req.IdempotencyKey),
 TaskQueue: TaskQueue,
 WorkflowIDConflictPolicy: client.WorkflowIDConflictPolicyFail,
 }
 return tc.client.ExecuteWorkflow(ctx, opts, ProvisionWalletWorkflow, req)
}

```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Workflow-start test	Direct invocation, task queue alignment, operation id correlation	Start ProvisionWalletWorkflow → verify deterministic ID matches operation record

No-Nexus proof	Nexus disabled	Verify <code>wallet.nexus.routing.enabled = false</code> in config; no Nexus service handler registration
Task queue evidence	Worker polls expected task queue	Local worker smoke test confirms <code>wallet-operations</code> task queue polling

## WBS-BASE-15: Worker Bootstrap Boundary

Time: 2h | MR: MR-BASE-04 | Dependencies: WBS-BASE-14, AC-02/G1-G3, AC-08/G3

### Files to Create/Modify

File	Action	Description
<code>cmd/worker/main.go</code>	CREATE	New worker endpoint registering workflows, activities, adapters, retry policies
<code>internal/temporal/worker.go</code>	CREATE	Worker configuration: workflow/activity registration, task queue, retry policies

<code>internal/temporal/worker_test.go</code>	CREATE	Local worker smoke test
-----------------------------------------------	--------	-------------------------

## Key Code — Worker Entrypoint

```
// cmd/worker/main.go
package main

import (
 "log"
 "go.temporal.io/sdk/worker"
 "go.temporal.io/sdk/client"
 "github.com/Polity_MVP/wallet-infrastructure-service/internal/temporal"
 "github.com/Polity_MVP/wallet-infrastructure-service/internal/adapters/dfns"

 "github.com/Polity_MVP/wallet-infrastructure-service/internal/adapters/safeheron"
 "github.com/Polity_MVP/wallet-infrastructure-service/internal/adapters/fireblocks"
 "github.com/Polity_MVP/wallet-infrastructure-service/internal/contract"
 "github.com/Polity_MVP/wallet-infrastructure-service/config"
)

func main() {
 cfg := config.Load()
 temporalClient, err := client.Dial(client.Options{
 HostPort: cfg.TemporalHostPort,
 Namespace: temporal.Namespace,
 })
 if err != nil {
 log.Fatalf("failed to connect to Temporal: %v", err)
 }

 registry := contract.NewAdapterRegistry()
 registry.Register(&dfns.DFNSAdapter{}) // wraps existing DFNS service
 registry.Register(&safeheron.SafeheronAdapter{}) // wraps existing
Safeheron service
 registry.Register(&fireblocks.DisabledFixture{}) // disabled fixture
}
```



```

w := worker.New(temporalClient, temporal.TaskQueue, worker.Options{})

// Register workflows
w.RegisterWorkflow(temporal.ProvisionWalletWorkflow)
w.RegisterWorkflow(temporal.RegisterAccountWorkflow)

// Register Tier 3 Canonical Activities
w.RegisterActivity(temporal.FetchServiceCredentials)
w.RegisterActivity(temporal.CreateProviderAccount)
w.RegisterActivity(temporal.StoreCredentials)
w.RegisterActivity(temporal.CreateWallet)
w.RegisterActivity(temporal.AddAsset)
w.RegisterActivity(temporal.CommitProvisioning)
w.RegisterActivity(temporal.ValidateProviderConnectivity)
w.RegisterActivity(temporal.RevokeProviderAccount)
w.RegisterActivity(temporal.RevokeStoredCredentials)
w.RegisterActivity(temporal.ReconcileWallets)

if err := w.Run(worker.InterruptCh()); err != nil {
 log.Fatalf("worker failed: %v", err)
}
}

```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Local worker proof	Configured worker polls expected task queue	Start local Temporal with Docker Compose, run worker, confirm task queue polling logs
Configuration sample	ID/task-queue convention	Verify deterministic workflow IDs use <code>wallet-{providerID}-{opType}-{idempotencyKey}</code> format

# WBS-BASE-16: Workflow Determinism and History Hygiene

Time: 2h | MR: MR-BASE-04 | Dependencies: WBS-BASE-14, AC-03/G3-G4, AC-08/G3

## Files to Create/Modify

File	Action	Description
<code>internal/temporal/workflows.go</code>	MODIFY	Ensure workflows carry only non-secret identifiers
<code>internal/temporal/workflow_test.go</code>	CREATE	Workflow tests, replay tests, history-scan
<code>internal/temporal/history_audit.go</code>	CREATE	History inspection tool for secret leakage

## Key Code — Deterministic Workflow

```
// internal/temporal/workflows.go
package temporal

// Workflow inputs MUST contain only non-secret identifiers.
// No credentials, private keys, signing payloads, or provider SDK clients.

type ProvisionWalletRequest struct {
 OperationID string `json:"operation_id"`
 ProviderID string `json:"provider_id"`
}
```

```

 OperationType string `json:"operation_type"`
 UserID string `json:"user_id"`
 IdempotencyKey string `json:"idempotency_key"`
 CorrelationID string `json:"correlation_id"`
 ChainID string `json:"chain_id"`
 // NO credentials, NO secrets, NO provider SDK objects
}

func ProvisionWalletWorkflow(ctx workflow.Context, req ProvisionWalletRequest)
error {
 // Workflow logic is deterministic:
 // - No secrets in workflow state
 // - No SDK clients in workflow state
 // - No file I/O or database writes in workflow
 // - All side effects are in Activities
 // - Decisions based only on operation IDs and non-secret context

 ao := workflow.ActivityOptions{
 StartToCloseTimeout: time.Minute * 5,
 RetryPolicy: &temporal.RetryPolicy{
 InitialInterval: time.Second * 5,
 BackoffCoefficient: 2.0,
 MaximumAttempts: 3,
 NonRetryableErrorTypes: []string{
 "PROVIDER_VALIDATION",
 "PROVIDER_AUTH_FAILURE",
 "PROVIDER_NOT_FOUND",
 "SENSITIVE_OPERATION_BLOCKED",
 },
 },
 }

 ctx = workflow.WithActivityOptions(ctx, ao)

 // Step 1: Fetch credentials (Activity - side effect)
 var credResp VaultCredentialResponse
 if err := workflow.ExecuteActivity(ctx, FetchServiceCredentials,
req).Get(ctx, &credResp); err != nil {
 return err
 }

 // Step 2: Create provider account (Activity - side effect)
 // Step 3: Create wallet (Activity - side effect)
 // ... SAGA pattern with compensation registration

```

```
 return nil
}
```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Workflow test	Replay/history inspection	Test workflow with mock activities, replay history, verify deterministic
History scan	No secrets in workflow inputs	Inspect Temporal workflow history → verify no credentials, private keys, or provider SDK objects
Non-secret workflow inputs	Payloads contain only IDs and display-safe metadata	Verify ProvisionWalletRequest and RegisterAccountRequest structs contain no secret fields

## WBS-BASE-17: Activity and Vault Execution Boundary

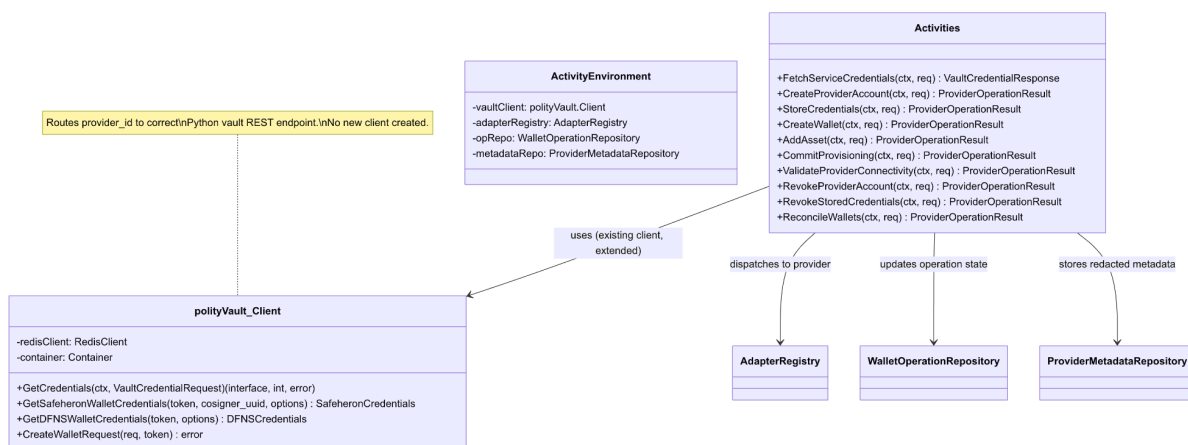
Time: 2h | MR: MR-BASE-04 | Dependencies: WBS-BASE-08, WBS-BASE-16, AC-03/G3-G4, G4 Vault readiness

### Files to Create/Modify



File	Action	Description
<code>internal/temporal/activities.go</code>	CREATE	All 9 Tier 3 Canonical Activities
<code>internal/temporal/activities_test.go</code>	CREATE	Activity tests, GetCredentials routing tests, redaction evidence
<code>providers/polityVault/client.go</code>	MODIFY	(Already extended in WBS-BASE-08 with <code>GetCredentials</code> method)

## Class Diagram — Activity Boundary



## Key Code — Activity Pattern

```

// internal/temporal/activities.go
package temporal

// Each Activity:

```



```
// 1. Fetches credentials through polityVault.Client.GetCredentials() (NOT
// Infisical directly)
// 2. Calls the selected ProviderAdapter
// 3. Maps canonical errors
// 4. Redacts evidence
// 5. Updates WALLET_OPERATIONS
// 6. Never serialises secrets into workflow history, logs, or fixtures
//
// IMPORTANT: Activities use the EXISTING polityVault.Client (extended in
// WBS-BASE-08)
// as the Vault gateway. No separate vault client is created. The Activity
// wrapper
// simply calls polityVault.Client.GetCredentials(ctx, req) and keeps the
// returned
// credential values in Activity-local scope only.

// FetchServiceCredentials is a Tier 3 Canonical Activity.
// It calls polityVault.Client.GetCredentials() which internally routes to the
// correct Python vault REST endpoint based on ProviderID:
// "dfns" → GET /api/v1/dfns/credentials
// "safeheron" → GET /api/v1/safeheron/credentials/<account_reference>/
// "fireblocks" → PROVIDER_NOT_FOUND error (no endpoint yet)
// unknown → PROVIDER_NOT_FOUND error
func (a *Activities) FetchServiceCredentials(
 ctx context.Context, req contract.VaultCredentialRequest,
) (contract.VaultCredentialResponse, error) {
 // Call the existing polityVault.Client which routes to the correct
 // endpoint.
 // The returned interface{} contains provider-specific credential objects
 // (vault.SafeheronCredentials or vault.DFNSCredentials) that live ONLY
 // in this Activity's scope – they are never returned to the workflow.
 rawCreds, statusCode, err := a.vaultClient.GetCredentials(ctx, req)
 if err != nil {
 // mapVaultError (defined in WBS-BASE-08) converts polityVault errors
 // to canonical ProviderOperationError with redaction.
 var opErr *contract.ProviderOperationError
 if errors.As(err, &opErr) {
 return contract.VaultCredentialResponse{}, opErr
 }
 return contract.VaultCredentialResponse{},
 &contract.ProviderOperationError{
 ProviderID: req.ProviderID,
 CanonicalClass: contract.PROVIDER_AUTH_FAILURE,
 Retryable: false,
 }
 }
}
```

```

 ProviderCode: "CREDENTIAL_FETCH_FAILED",
 ProviderMessageRedacted: "credential fetch failed",
 CorrelationID: req.CorrelationID,
 }
}

// The rawCreds are provider-specific (SafeheronCredentials or
DFNSCredentials).
// They stay in Activity scope. We return only non-secret metadata to the
workflow.
resp := contract.VaultCredentialResponse{
 ProviderID: req.ProviderID,
 AccountReference: req.AccountReference,
 LoadedAt: time.Now().UTC().Format(time.RFC3339),
 CacheHit: false, // Populated by polityVault.Client if cache
was used
}

// Store rawCreds in Activity-local variable for passing to
ProviderAdapter.
// They will be passed to the next Activity (CreateWallet, AddAsset, etc.)
// via Activity local state, NOT through workflow return values.
a.credentialCache[req.CorrelationID] = rawCreds

return resp, nil
}

```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Activity tests	Credentials loaded only at Activity time	Verify each Activity fetches credentials through Vault at start, not from workflow input

Log/history scans	No credential leakage	Scan Activity logs and Temporal history for secret patterns (private_key, jwt, api_key) → zero matches
Redaction evidence	Provider metadata redacted	Verify <code>ProviderOperationError.ProviderMessageRedacted</code> strips all secret material

## WBS-BASE-18: Retry and Compensation Evidence

Time: 2h | MR: MR-BASE-04 | Dependencies: WBS-BASE-11, WBS-BASE-17, AC-04/G6, AC-09/G6

### Files to Create/Modify

File	Action	Description
<code>internal/temporal/compensation.go</code>	CREATE	SAGA pattern: per-Activity compensation registration, reverse-order execution

<code>internal/temporal/retry.go</code>	CREATE	Retry matrix mapping canonical errors to Temporal retry policies
<code>internal/temporal/compensation_test.go</code>	CREATE	SAGA state tests, compensation evidence
<code>internal/temporal/retry_test.go</code>	CREATE	Retry limit tests, terminal state transitions

## Key Code — SAGA Pattern

```
// internal/temporal/compensation.go
package temporal

import "go.temporal.io/sdk/workflow"

// CompensationFn represents a per-Activity compensation action.
type CompensationFn func(ctx workflow.Context) error

// SAGA tracks compensations in reverse order.
type SAGA struct {
 compensations []CompensationFn
}

func (s *SAGA) Register(compensation CompensationFn) {
 s.compensations = append(s.compensations, compensation)
}

// Compensate executes compensations in reverse order (LIFO).
func (s *SAGA) Compensate(ctx workflow.Context) {
 for i := len(s.compensations) - 1; i >= 0; i-- {
 if err := s.compensations[i](ctx); err != nil {
```



```
// Log compensation failure, mark WALLET_OPERATIONS as
COMPENSATION_FAILED
// Do NOT panic – SAGA compensation is best-effort
}
}
}

// Retry policy mapping based on canonical error classes:
// PROVIDER_TIMEOUT, PROVIDER_RATE_LIMIT, PROVIDER_UNAVAILABLE → retry with
backoff
// PROVIDER_INTERNAL → max 3 attempts, then compensate
// PROVIDER_AUTH_FAILURE, PROVIDER_VALIDATION, PROVIDER_NOT_FOUND,
SENSITIVE_OPERATION_BLOCKED → non-retryable
// PROVIDER_CONFLICT → CONFLICT_IDEMPOTENT_SUCCESS (no retry)
// PROVIDER_PENDING → polling status, not a retry error
```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Retry matrix tests	Retry limits, terminal state transitions	Map each ErrorClass to correct retry behaviour
SAGA state tests	Step-status tracking, idempotency keys, reverse-order compensation	Simulate workflow with downstream failure → verify compensations execute in reverse order
Compensation evidence	Compensation fields, evidence references	Test that compensation results are recorded in WALLET_OPERATIONS



# WBS-BASE-19: Unified Wallet API Skeleton or Bridge

Time: 2h | MR: MR-BASE-05 | Dependencies: WBS-BASE-04, WBS-BASE-09..12, AC-07/G5-G6

## Files to Create/Modify

File	Action	Description
<code>internal/rest/api/v1/api.go</code>	CREATE	New unified API controller under <code>/api/wallets/v1/*</code>
<code>internal/rest/router/router.go</code>	MODIFY	Add unified route group preserving legacy routes
<code>internal/rest/handler/wallet_handler.go</code>	CREATE	Handler methods that orchestrate through <code>WalletOperationsService</code>

## Key Code — Unified API Routes

```
// internal/rest/router/router.go - ADD unified route group
// New unified routes (behind feature flag `wallet.api.v1.enabled`):

// GET /api/wallets/v1/ → ListWallets (provider-neutral)
// GET /api/wallets/v1/:id → GetWallet (provider-neutral)
// POST /api/wallets/v1/ → CreateWallet (starts ProvisionWalletWorkflow)
```

```
// PUT /api/wallets/v1/:id/rename → RenameWallet
// PUT /api/wallets/v1/:id/archive → ArchiveWallet
// GET /api/wallets/v1/:id/assets → ListAssets (provider-neutral)
// POST /api/wallets/v1/:id/assets → AddAsset (starts workflow)
// GET /api/wallets/v1/:id/addresses → GetAssetAddresses
// GET /api/wallets/v1/:id/status → GetWalletStatus

// All mutating endpoints require an idempotency_key in the request body.
// The caller is responsible for generating a unique idempotency_key (UUID v4).
// Duplicate requests with the same (provider_id, operation_type,
idempotency_key)
// return the existing WALLET_OPERATIONS record as CONFLICT_IDEMPOTENT_SUCCESS.

// Legacy routes REMAIN ACTIVE until compatibility bridge is accepted:
// GET /api/v1/wallets/dfns → Bridge to unified handler
(dfns provider)
// GET /api/v1/wallets/safeheron → Bridge to unified handler
(safeheron provider)
// ... all other legacy routes unchanged for now
```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Unified route skeleton	New /api/wallets/v1/ * routes registered	Run router, enumerate routes, verify unified endpoints exist
Handler tests	Existing reads remain stable	Test legacy /wallets/dfns and /wallets/safeheron still return correct data through bridge
Sensitive -operation routes	Existing sensitive-oper	Test that payment, signing, broadcast endpoints behind kill-switch return blocked

fail closed	ation routes fail closed	
-------------	--------------------------	--

## WBS-BASE-20: Route Fate and Deprecation Matrix

Time: 2h | MR: MR-BASE-05 | Dependencies: WBS-BASE-01, WBS-BASE-19, AC-07/G5-G6

### Files to Create/Modify

File	Action	Description
<code>internal/contract/route_fate.go</code>	CREATE	Route fate matrix: preserve, wrap, deprecate, disable, remove
<code>internal/rest/router/router.go</code>	MODIFY	Apply fate decisions: disable sensitive routes, wrap lifecycle routes
<code>internal/rest/router/fate_test.go</code>	CREATE	Before/after route inventory, runtime checks

### Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Route fate matrix	Classify every route	Table with route, method, current handler, fate, reason, rollback path
OpenAPI delta	Before/after API spec comparison	Generate OpenAPI from router, compare legacy vs. unified
Runtime route proof	Sensitive paths unreachable at runtime	Start server, hit <code>/api/v1/payment/*</code> , <code>/api/v1/walletconnect/*</code> , verify blocked response

## WBS-BASE-21: Sensitive-Operation Negative Capability Closure

Time: 2h | MR: MR-BASE-05 | Dependencies: WBS-BASE-07, WBS-BASE-20, G5 sensitive-operation closure

### Files to Create/Modify

File	Action	Description
------	--------	-------------

<code>internal/rest/router/router.go</code>	MODIFY	Register kill-switch middleware for sensitive routes
<code>internal/contract/sensitive_paths.go</code>	MODIFY	Evolve from static inventory (WBS-BASE-01) to runtime enforcement — see below
<code>internal/rest/api/api.go</code>	MODIFY	Remove or disable payment, signing, broadcast, WalletConnect, scan handlers
<code>scripts/sensitive_operation_scan.sh</code>	CREATE	Static scan script
<code>internal/rest/negative_test.go</code>	CREATE	Negative capability tests for each sensitive operation

## Modification Detail —

### `internal/contract/sensitive_paths.go`

In WBS-BASE-01 this file was created as a static inventory document: a `SensitivePathRegistry` slice of `RouteEntry` structs documenting route classification, with a comment explicitly stating "inventory only, no runtime activation." WBS-BASE-21 evolves this file from a passive inventory into an active runtime enforcement mechanism by adding:



1. `SensitivePathEnforcer` struct and `Enforce()` method — iterates over `SensitivePathRegistry` at server startup and ensures each route with `Fate: "disable"` is either removed from the router, replaced with a canonical rejection handler, or gated behind the sensitive-operation kill switch middleware. This replaces the passive comment with concrete enforcement logic.
2. `IsSensitiveOperation()` function — exported function that checks whether a given `(method, path)` pair is classified as a sensitive operation. Used by the kill-switch middleware in `router.go` to block sensitive operations at runtime before they reach the handler.
3. `RegisterDisabledRouteHandlers()` function — called from `router.go` during route registration to mount canonical rejection handlers for every `Fate: "disable"` route. Each handler returns a `ProviderOperationError` with `CanonicalClass: PROVIDER_VALIDATION` and `BlockedByGate: "sensitive-operation-kill-switch"`, consistent with the disabled-operation responses defined in WBS-BASE-04.
4. Updated `RouteEntry` struct — add `DisabledHandler` field of type `func(ctx *fasthttp.RequestCtx)` for routes that need a specific disabled response (e.g. payment routes return a different body than webhook routes), and `KillSwitchKey` field that maps to the feature flag controlling this route's sensitive-operation kill switch.
5. `VerifySensitivePathsAbsence()` function — test helper that performs a static code scan over registered routes after startup, confirming no handler that reaches `MakePayment`, `BroadcastTransaction`, `PersonalSign`, `WalletConnect`, or `Scan` logic is reachable for any route with `Fate: "disable"`. Used by `negative_test.go` acceptance evidence.

```
// internal/contract/sensitive_paths.go — EVOLVED from WBS-BASE-01 to runtime enforcement
```

```
package contract
```

```
import "github.com/valyala/fasthttp"
```

```
type RouteEntry struct {
 Method string
 Path string
 Category string
 Fate string // "preserve", "wrap", "disable", "deprecate",
 "remove"
```

```

 KillSwitchKey string // feature flag key controlling this route's kill
switch
 DisabledHandler fasthttp.RequestHandler // canonical rejection handler for
disabled routes
}

// SensitivePathRegistry documents all wallet routes classified by fate.
// In WBS-BASE-01 this was inventory-only. In WBS-BASE-21 it is enforced at
runtime.
var SensitivePathRegistry = []RouteEntry{
 {Method: "POST", Path: "/api/v1/payment/dfns", Category: "payment", Fate:
"disable", KillSwitchKey: "wallet.payment.enabled", DisabledHandler:
DisabledPaymentHandler},
 {Method: "POST", Path: "/api/v1/payment/safeheron", Category: "payment",
Fate: "disable", KillSwitchKey: "wallet.payment.enabled", DisabledHandler:
DisabledPaymentHandler},
 {Method: "POST", Path: "/api/v1/payment/resolve", Category: "payment",
Fate: "disable", KillSwitchKey: "wallet.payment.enabled", DisabledHandler:
DisabledPaymentHandler},
 {Method: "POST", Path: "/api/v1/walletconnect/invoke", Category:
"walletconnect", Fate: "disable", KillSwitchKey:
"wallet.walletconnect.enabled", DisabledHandler: DisabledWalletConnectHandler},
 {Method: "POST", Path: "/api/v1/wallet/scan/dfns", Category: "scan", Fate:
"disable", KillSwitchKey: "wallet.scan.enabled", DisabledHandler:
DisabledScanHandler},
 {Method: "POST", Path: "/api/v1/wallet/scan/safeheron", Category: "scan",
Fate: "disable", KillSwitchKey: "wallet.scan.enabled", DisabledHandler:
DisabledScanHandler},
 {Method: "POST", Path: "/api/v1/dfns/notification", Category: "webhook",
Fate: "disable", KillSwitchKey: "wallet.webhook.dfns.enabled", DisabledHandler:
DisabledWebhookHandler},
 {Method: "POST", Path: "/api/v1/safeheron/notification", Category:
"webhook", Fate: "disable", KillSwitchKey: "wallet.webhook.safeheron.enabled",
DisabledHandler: DisabledWebhookHandler},
}

// IsSensitiveOperation checks whether (method, path) is classified as
sensitive.
func IsSensitiveOperation(method, path string) bool {
 for _, entry := range SensitivePathRegistry {
 if entry.Method == method && entry.Path == path && entry.Fate ==
"disable" {
 return true
 }
 }
}

```

```

 }
 return false
}

// RegisterDisabledRouteHandlers mounts canonical rejection handlers for all
// disabled routes. Called from router.go during route registration.
func RegisterDisabledRouteHandlers(registry *SensitivePathRegistry) {
 for _, entry := range SensitivePathRegistry {
 if entry.Fate == "disable" {
 registry.RegisterDisabled(entry.Method, entry.Path,
 entry.DisabledHandler)
 }
 }
}

```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Static scan	Code scan of WIS for sensitive paths	Script that searches for <code>MakePayment</code> , <code>BroadcastTransaction</code> , <code>PersonalSign</code> , <code>WalletConnect</code> , <code>Scan</code> in handler code paths
Runtime route inventory	No reachable sensitive-operation path without exception	Start server, enumerate all routes, verify payment/signing/broadcast/WalletConnect/scan/admin paths return non-2xx

Negative operation tests	Each sensitive operation returns disabled/locked response	Test each disabled endpoint returns canonical error
Provider-side evidence	DFNS and Safeheron provider evidence for non-custodial controls	Documentation from providers confirming Polity lacks key share

## WBS-BASE-22: Dynamic/Auth/Frontend Compatibility

Time: 1h | MR: MR-BASE-05 | Dependencies: WBS-BASE-19, AC-09/G6, AC-10/G6-G7

### Files to Examine/Modify



File	Action	Description
<code>frontend/src/apps/cabinet/features/Wallet/</code>	Examine	Verify Dynamic sign-in, wallet linking, wallet display compatibility
<code>frontend/src/apps/cabinet/features/Wallet/api/rtq/base.ts</code>	Examine	RTK Query endpoints for wallet API
<code>frontend/src/apps/cabinet/features/Wallet/pages/InstallWallet/lib/useWalletFlow.ts</code>	Examine	Wallet creation wizard flow
<code>internal/rest/api/api.go</code>	Examine	Response shape compatibility for legacy reads
<code>frontend/src/apps/cabinet/features/Wallet/types/base.ts</code>	MODIFY	Update <code>WalletProvider</code> type to include <code>'fireblocks'</code> as disabled

## Key Code — Frontend Provider Type Update

```
// frontend/src/apps/cabinet/features/Wallet/types/base.ts
// Current: type WalletProvider = 'dfns' | 'safeheron';
// Target:
```



```
type WalletProvider = 'dfns' | 'safeheron' | 'fireblocks';

// 'fireblocks' is display-safe disabled identity only:
// - Backend returns capability state = DISABLED for fireblocks
// - Frontend shows "Coming Soon" or badge for fireblocks
// - No WalletConnect, signing, payment, or broadcast is reachable
```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Dynamic sign-in compatibility	Verify sign-in works with new API	E2E test: Dynamic V4 sign-in → wallet display still works
Wallet display compatibility	Response samples	Verify unified API returns compatible wallet data shapes
Supported-network tests	Non-baseline chains stay disabled	Test that requesting unapproved chain returns <code>UNSUPPORTED</code>
Compatibility note	Veriff/Lukka non-interference	Document that wallet infrastructure changes do not affect identity verification or risk analytics

## WBS-BASE-23: Callback/Webhook Envelope Boundary

Time: 2h | MR: MR-BASE-05 | Dependencies: WBS-BASE-20, AC-07/G5-G6

## Files to Create/Modify

File	Action	Description
<code>internal/contract/callback_envelope.go</code>	CREATE	Common callback envelope with raw-body preservation, signature verification result, event type, event id, operation id
<code>internal/rest/handler/callback_handler.go</code>	CREATE	Unified callback handler with fail-closed logic
<code>internal/rest/handler/callback_handler_test.go</code>	CREATE	Invalid, duplicate, unknown, stale, and sensitive callback tests

## Key Code — Callback Envelope

```
// internal/contract/callback_envelope.go
package contract

type CallbackEnvelope struct {
 RawBody []byte
 SignatureVerified bool
 SignatureResult string // "valid", "invalid", "missing"
 ProviderEventType string
 ProviderEventID string
 TargetOperationID string
 RedactedPayload map[string]interface{}
 Duplicate bool
 IgnoredOrDeferred bool
 ProviderID string
 Timestamp string
}

// Processing rules:
// 1. Invalid/missing signature → reject before mutation, log as security event
// 2. Duplicate provider event id → do not duplicate state transition
// 3. Stale timestamp or replay-window breach → reject or quarantine
// 4. Unknown provider or operation → reject or quarantine
// 5. Sensitive or deferred event → record as blocked/deferred, no activation
```

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Envelope tests	Invalid, duplicate, unknown, stale, and sensitive callbacks fail closed	Test each callback scenario returns 403/422 before mutation

No mutation before verification	No state mutation before signature verification	Verify callback handler checks signature before any database write
Provider handoff note	Provider-specific signatures and subscriptions are provider RFP scope	Document that DFNS HMAC and Safeheron callback verification remain in current handlers until provider RFPs migrate to envelope

## WBS-BASE-24: Traceability and Evidence Index

Time: 2h | MR: MR-BASE-06 | Dependencies: All prior rows, G0-G7

### Files to Create/Modify

File	Action	Description
<code>docs/evidence/traceability_matrix.csv</code>	CREATE	CSV mapping: WBS → FR/NFR → TOR gate → MR-BASE → test evidence → rollback → owner → blocked condition

<code>docs/evidence/evidence_index.md</code>	CREATE	Index of all evidence packs per MR
----------------------------------------------	--------	------------------------------------

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Traceability matrix	CSV-ready mapping of every WBS row to requirements, MR, test, rollback, owner	Reviewable CSV with all 26 WBS rows mapped
Evidence index	Hierarchical index per MR-BASE package	Cross-reference to test outputs, migration logs, route scans
Blocked-condition register	Unresolved blockers with owner and expiry	Capture any dependency gates that prevented evidence completion

## WBS-BASE-25: Runbook and Provider-Start Handoff

Time: 2h | MR: MR-BASE-06 | Dependencies: All implementation rows, AC-10/G6-G7

### Files to Create/Modify



File	Action	Description
<code>docs/runbook.md</code>	CREATE	Rollback procedures, provider-start prerequisites, open blockers, handoff rules
<code>docs/provider_start_checklist.md</code>	CREATE	Per-provider (Fireblocks, DFNS, Safeheron) checklist of what they must consume from base

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
Runbook	Rollback procedures for each migration and route change	Step-by-step rollback commands
Provider-start checklist	Fireblocks, DFNS, Safeheron RFPs consume same base artefact	Document: registry slot, disabled fixture, capability contract, vault boundary, temporal readiness, evidence pack
Dependency closure note	Remaining blockers	List any unresolved gates preventing provider RFP start

# WBS-BASE-26: Final QA and Release-Boundary Review

Time: 3h | MR: MR-BASE-06 | Dependencies: WBS-BASE-24..25, G6-G7 closure

## Tasks

1. Run `go test ./...` for WIS — all passing
2. Run `golangci-lint run ./...` — zero errors or approved exceptions
3. Run `polity_tests` integration evidence where WIS, Vault, Auth Gateway, or frontend wallet display is affected
4. Route scan — enumerate all routes, verify sensitive-operation closure
5. Secret scan — verify no secrets in workflow history, logs, fixtures, or database
6. Release-boundary review — confirm R1.3.2 boundary is unchanged (no provider custody, signing, raw broadcast, payment, WalletConnect, or chain scope expansion)

## Acceptance Evidence

Evidence	RFP Spec	Validated Assessment
QA log	Command output from <code>go test</code> , <code>golangci-lint</code> , integration tests	Full test output captured
Release-boundary note	R1.3.2 boundary unchanged	Explicit statement that no sensitive operation scope has expanded

Final review memo	Reviewer sign-off	Polity Programme Lead sign-off on evidence pack
-------------------	-------------------	-------------------------------------------------

## Summary: Files Created/Modified

### New Files (internal/contract package)

File	WBS
<code>internal/contract/provider_adapter.go</code>	WBS-BASE-03, 04
<code>internal/contract/provider_credentials.go</code>	WBS-BASE-03
<code>internal/contract/wallet_operations.go</code>	WBS-BASE-03
<code>internal/contract/provider_capabilities.go</code>	WBS-BASE-05
<code>internal/contract/capabilities.go</code>	WBS-BASE-05
<code>internal/contract/errors.go</code>	WBS-BASE-05
<code>internal/contract/registry.go</code>	WBS-BASE-06
<code>internal/contract/gates.go</code>	WBS-BASE-07

<code>internal/contract/vault_gateway.go</code>	WBS-BASE-08
<code>internal/contract/sensitive_paths.go</code>	WBS-BASE-01, 21
<code>internal/contract/route_fate.go</code>	WBS-BASE-20
<code>internal/contract/callback_envelope.go</code>	WBS-BASE-23
<code>internal/contract/chains.go</code>	WBS-BASE-10
<code>internal/contract/provider.go</code>	WBS-BASE-09

## New Files (internal/adapter package)

File	WBS
<code>internal/adapter/dfns/adapter.go</code>	WBS-BASE-04
<code>internal/adapter/safeheron/adapter.go</code>	WBS-BASE-04
<code>internal/adapter/fireblocks/disabled.go</code>	WBS-BASE-04

## New Files (internal/temporal package)

File	WBS
------	-----

<code>internal/temporal/client.go</code>	WBS-BASE-14
<code>internal/temporal/config.go</code>	WBS-BASE-14
<code>internal/temporal/workflows.go</code>	WBS-BASE-16
<code>internal/temporal/worker.go</code>	WBS-BASE-15
<code>internal/temporal/activities.go</code>	WBS-BASE-17
<code>internal/temporal/activities_test.go</code>	WBS-BASE-17
<code>internal/temporal/compensation.go</code>	WBS-BASE-18
<code>internal/temporal/retry.go</code>	WBS-BASE-18

## New Files (models and storage)

File	WBS
<code>internal/models/provider/provider.go</code>	WBS-BASE-09
<code>internal/models/provider/account.go</code>	WBS-BASE-09
<code>internal/models/wallet/wallet.go</code>	WBS-BASE-09
<code>internal/models/wallet/asset.go</code>	WBS-BASE-09



<code>internal/models/operation/operation.go</code>	WBS-BASE-11
<code>internal/models/provider/metadata.go</code>	WBS-BASE-09
<code>internal/repository/operation_repository.go</code>	WBS-BASE-12
<code>internal/storage/seed.go</code>	WBS-BASE-10
<code>internal/storage/compatibility.go</code>	WBS-BASE-10
<code>internal/storage/rollback.go</code>	WBS-BASE-13

## New Files (API and routes)

File	WBS
<code>internal/rest/api/v1/api.go</code>	WBS-BASE-19
<code>internal/rest/handler/wallet_handler.go</code>	WBS-BASE-19
<code>internal/rest/handler/callback_handler.go</code>	WBS-BASE-23
<code>cmd/worker/main.go</code>	WBS-BASE-15

## Modified Files

File	WBS
<code>internal/rest/router/router.go</code>	WBS-BASE-19, 20, 21
<code>internal/rest/api/api.go</code>	WBS-BASE-21
<code>internal/service/interface.go</code>	WBS-BASE-04
<code>internal/storage/migrations/migrations.go</code>	WBS-BASE-09
<code>providers/polityVault/client.go</code>	WBS-BASE-08, 17
<code>providers/polityVault/client_test.go</code>	WBS-BASE-08
<code>providers/polityVault/mock.go</code>	WBS-BASE-08
<code>flagger/flagger.go</code>	WBS-BASE-07
<code>config/config.go</code>	WBS-BASE-07
<code>frontend/.../types/base.ts</code>	WBS-BASE-22

## Documentation Files

File	WBS
<code>docs/evidence/traceability_matrix.csv</code>	WBS-BASE-24

<code>docs/evidence/evidence_index.md</code>	WBS-BASE-24
<code>docs/runbook.md</code>	WBS-BASE-25
<code>docs/provider_start_checklist.md</code>	WBS-BASE-25
<code>docs/rollback-notes.md</code>	WBS-BASE-13
<code>scripts/sensitive_operation_scan.sh</code>	WBS-BASE-21

## Key Dependencies Beyond wallet-infrastructure-service

Dependency	Impact	WBS
<code>polity_vault/</code> (Python service)	No changes needed for base RFP. The Python vault REST API ( <code>/api/v1/dfns/credentials</code> , <code>/api/v1/safeheron/credentials/&lt;uuid&gt;/</code> , <code>/api/v1/creation-request/</code> ) and the <code>vault_backend/client.py</code> Infisical integration remain unchanged. The Go client routes <code>provider_id</code> to the correct REST endpoint internally. A generic <code>/api/v1/credentials</code> endpoint may be added in a provider RFP (Fireblocks) if needed.	WBS-BASE-08

<code>frontend/</code>	<code>WalletProvider</code> type update to include <code>'fireblocks'</code> as disabled	WBS-BA SE-22
<code>auth-gateway/</code>	JWT claims may need <code>provider_id</code> extension for gate evaluation	WBS-BA SE-07
<code>docker-compose.yml</code>	Add Temporal service for local development/testing	WBS-BA SE-14, 15
<code>go.mod</code>	Add <code>go.temporal.io/sdk</code> dependency	WBS-BA SE-14